



Universidade Federal de Santa Catarina – UFSC
DEC – Engenharia de Computação

Linguagem de Descrição de Hardware – Unidade 2

Dr. Ney Laert Vilar Calazans

Última alteração: 10/10/2025

❖ Baseado em materiais originais dos Profs. **Ney Calazans**, **Fernando Moraes (PUCRS)**, e **Rafael Garibotti (PUCRS)**

TUTORIAL 1: CAPTURA DE PROJETO E SIMULAÇÃO DE HARDWARE (XILINX ISIM)

USANDO O SIMULADOR ISIM DA XILINX (ISE 14.7)

➤ Criando os arquivos do projeto

1. Criar um diretório de trabalho, por exemplo `halfadder`
2. Criar os dois arquivos fonte, com extensão VHDL

✓ `halfadd.vhd`

```
library IEEE;
use IEEE.std_logic_1164.all;

entity halfadd is
port (A, B: in std_logic;
      Sum, Carry: out std_logic);
end halfadd;

architecture comp of halfadd is
begin
    Sum    <= A xor B;
    Carry <= A and B;
end comp;
```

✓ `halfadd_tb.vhd`

```
library IEEE;
use IEEE.std_logic_1164.all;

entity halfadd_tb is
end halfadd_tb;

architecture TB_ARCHITECTURE of halfadd_tb is
    signal aa, bb, soma, vaium : std_logic;
begin

    UUT : entity work.halfadd
        port map ( A => aa, B => bb,
                  Sum => soma, Carry => vaium );

    aa <= '0', '1' after 10 ns, '0' after 20 ns, '1' after 30 ns;

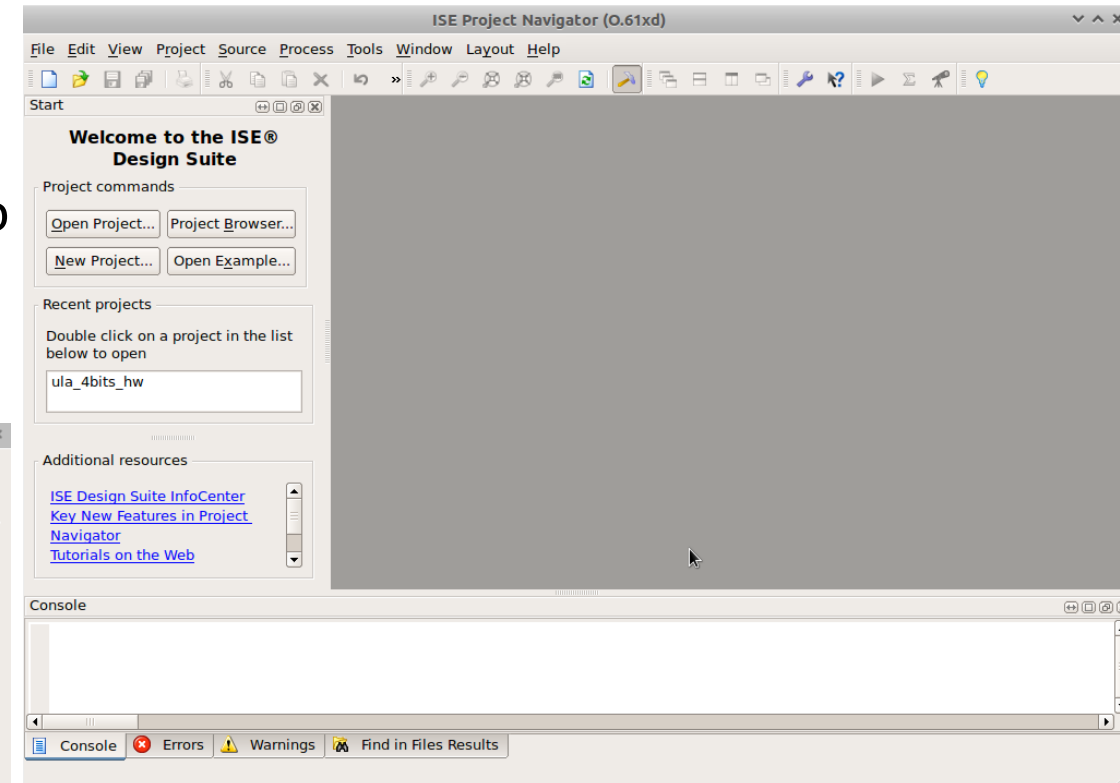
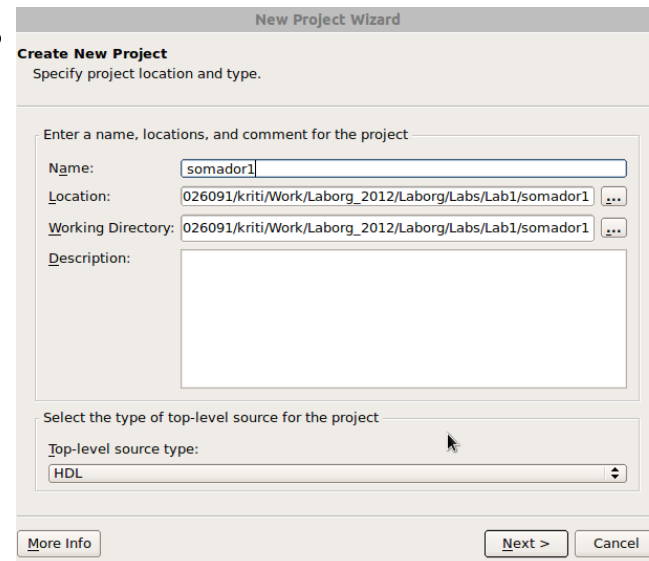
    bb <= '0', '1' after 20 ns;

end TB_ARCHITECTURE;
```

USANDO O SIMULADOR ISIM DA XILINX

➤ Criando um projeto no ambiente ISE para o FPGA da Nexys2 ou Nexys

1. Caso algum projeto seja aberto ao lançar o ISE, feche-o com a opção de menu **File→Close Project**
2. Crie um novo projeto de nome **halfadd** apertando o botão **New Project**. (Lembre: escolha um diretório no qual tens direito de escrita)
3. Depois de preencher os campos, clique no botão **Next**. Obs: **Não usem caracteres especiais nem no nome do projeto nem no nome do caminho** (acentos, brancos, etc.)



USANDO O SIMULADOR ISIM DA XILINX

4. Crie um projeto com as características mostradas no quadro se usar a placa **Nexys 2** com o dispositivo tamanho **1200**. Para o dispositivo tamanho **500** basta escolher o device **XC3S500E** ao invés do **XC3S1200E**, o resto fica igual. Dados importantes: escolha do dispositivo e do **ISE Simulator - ISIM** na opção **Simulator**. Depois clique no botão **Next**
5. No caso da placa **Nexys**, as escolhas são diferentes
 - **Spartan3** para a Family
 - **XC3S200** para o Device
 - **FT256** para o Package
 - **-4** para o Speed Grade

New Project Wizard

Project Settings
Specify device and project properties.

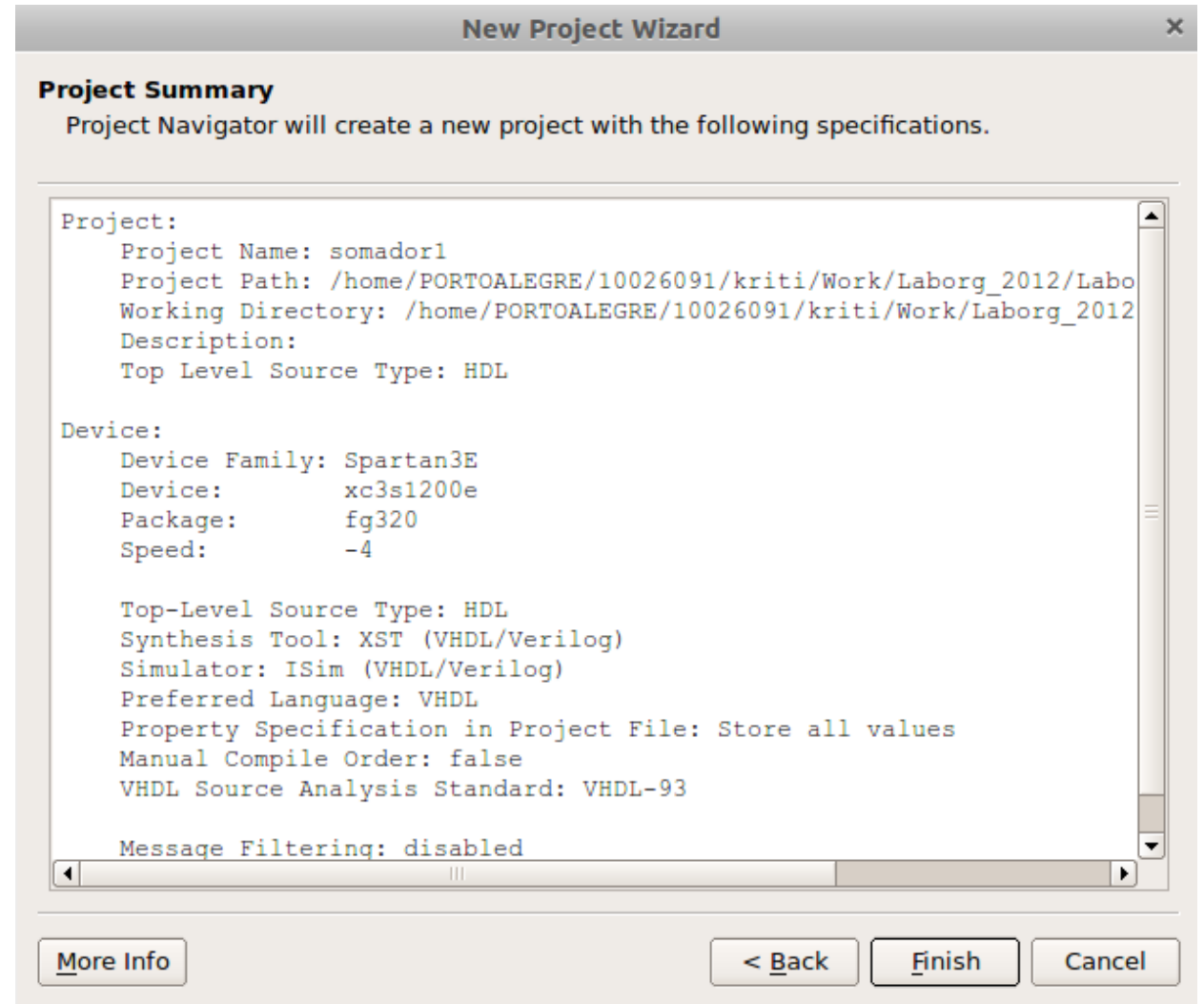
Select the device and design flow for the project

Product Category	All
Family	Spartan3E
Device	XC3S1200E
Package	FG320
Speed	-4
Top-Level Source Type	HDL
Synthesis Tool	XST (VHDL/Verilog)
Simulator	ISim (VHDL/Verilog)
Preferred Language	VHDL
Property Specification in Project File	Store all values
Manual Compile Order	☐
VHDL Source Analysis Standard	VHDL-93 Default: Unchecked

[More Info](#) [< Back](#) [Next >](#) [Cancel](#)

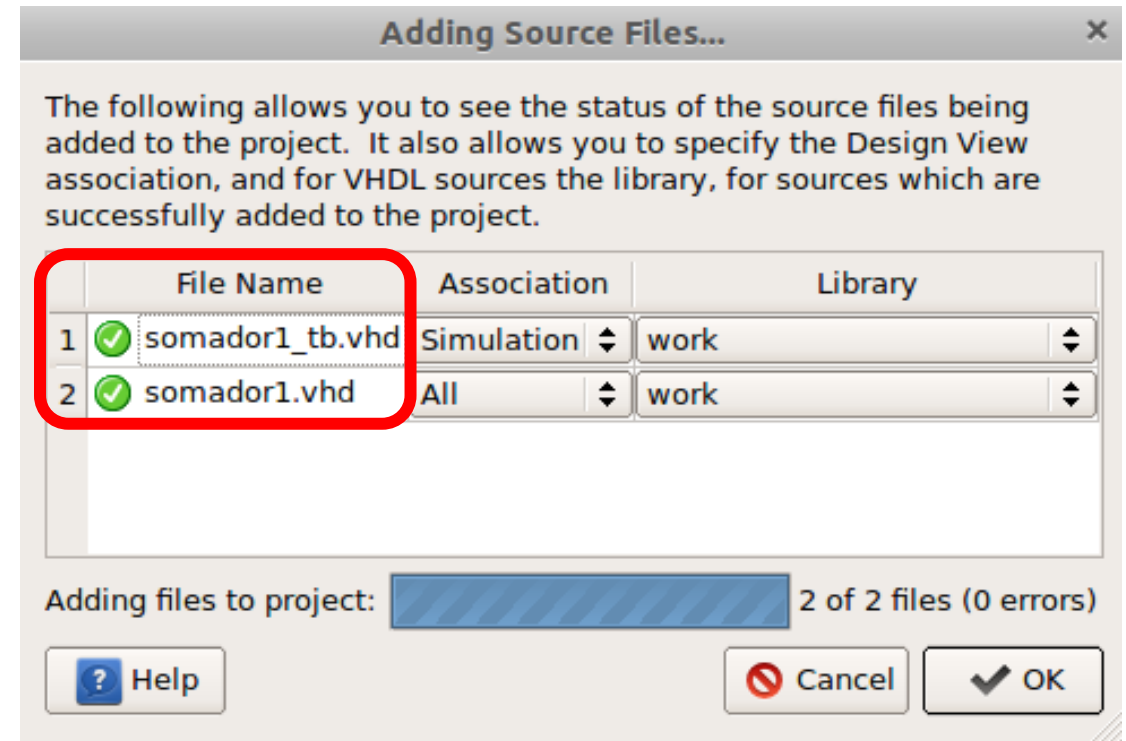
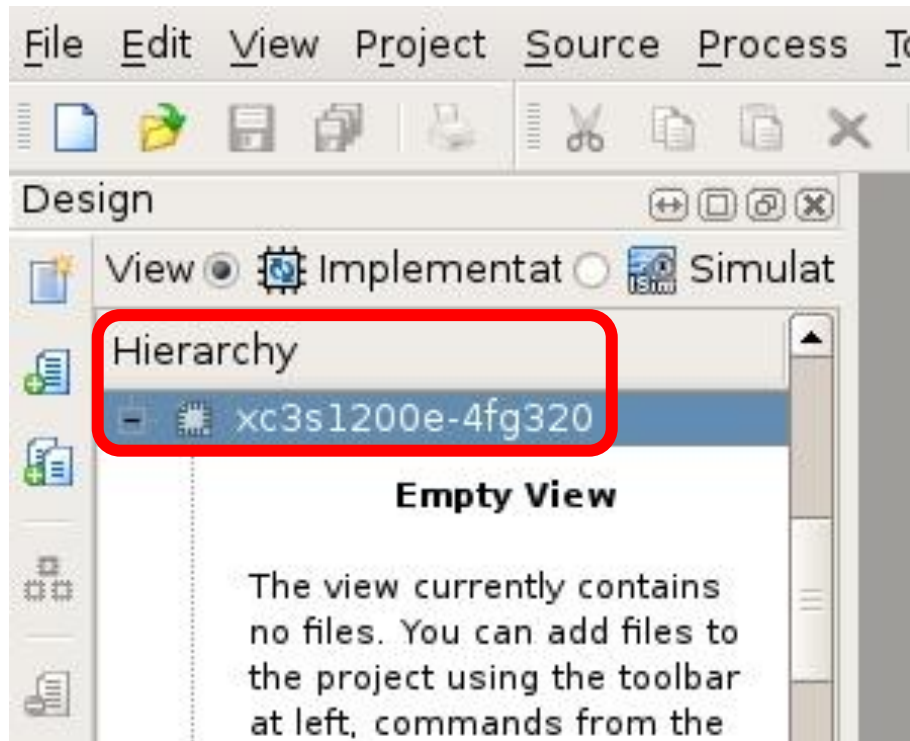
USANDO O SIMULADOR ISIM DA XILINX

5. Na janela final, confira os dados do projeto e depois simplesmente clique em **Finish**



USANDO O SIMULADOR ISIM DA XILINX

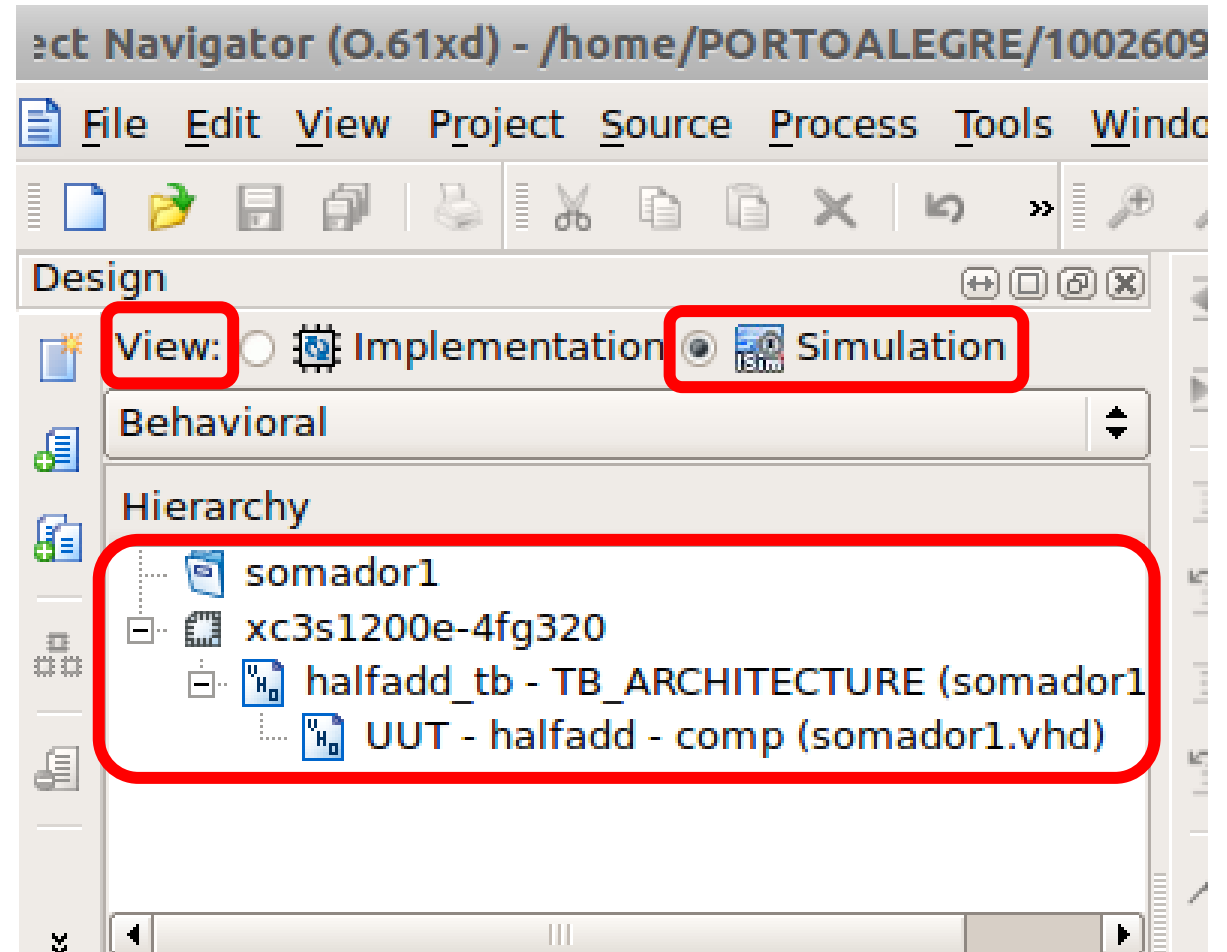
6. No ISE, clique com o botão direito no FPGA (Janela **Hierarchy**) – **Add Copy of Source**. Navegue na hierarquia e encontre os 2 arquivos VHDL citados respectivamente como **halfadd.vhd** e **halfadd_tb.vhd** (veja o slide 3 desta apresentação), em algum diretório acima do diretório principal do projeto (criado pelo ISE). A seguir, clique em **OK**



USANDO O SIMULADOR ISIM DA XILINX

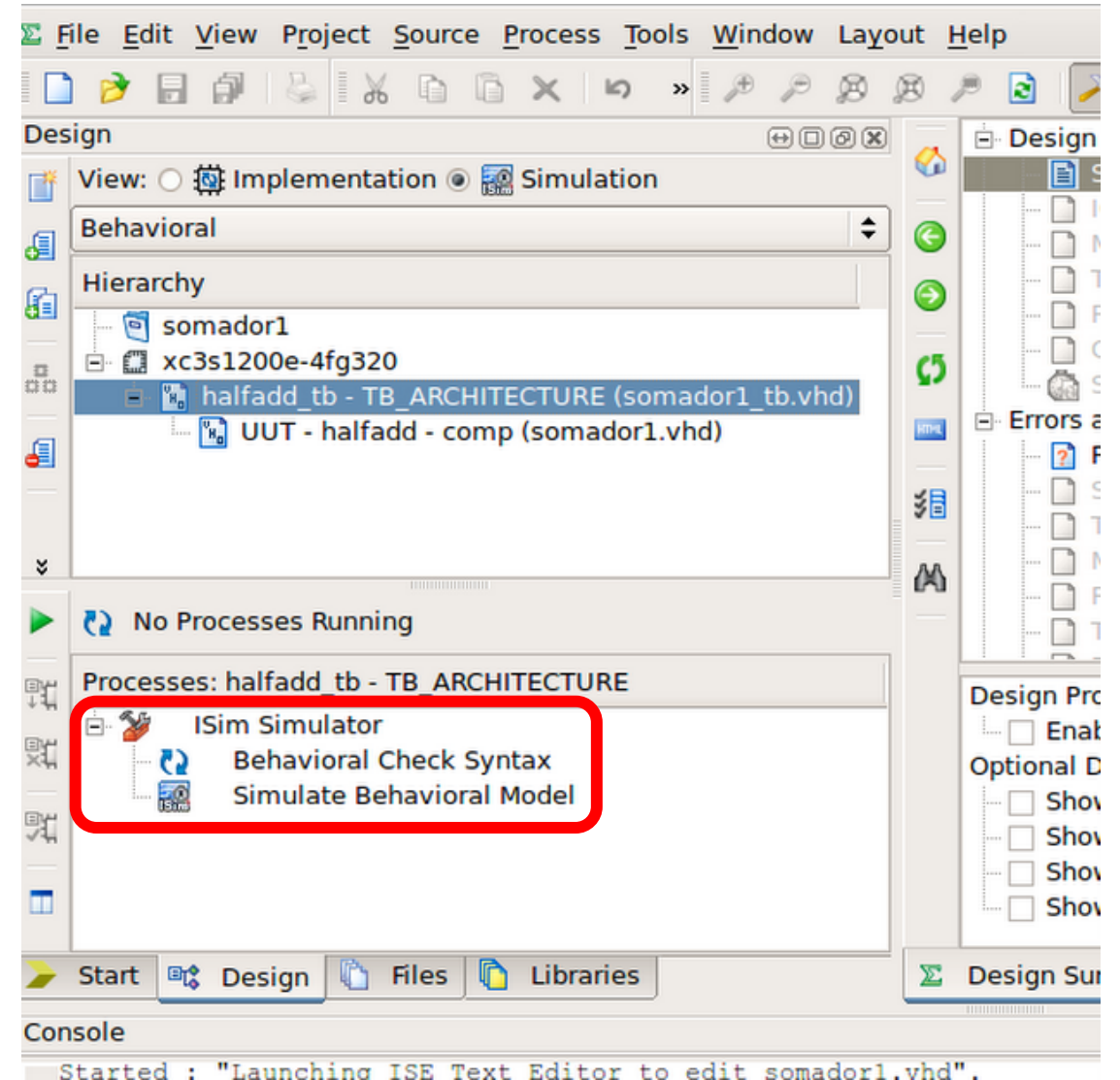
7. Note na janela anterior que o ambiente detectou corretamente os arquivos como sendo: um a ser usado apenas para simulação (o **testbench**, classificado como **Simulation**) e o outro como geral (**All**), significando que ele contém VHDL sintetizável

No Browser principal do projeto, escolha a visão de simulação (**View: Simulation**), pois por omissão a visão escolhida é a de implementação



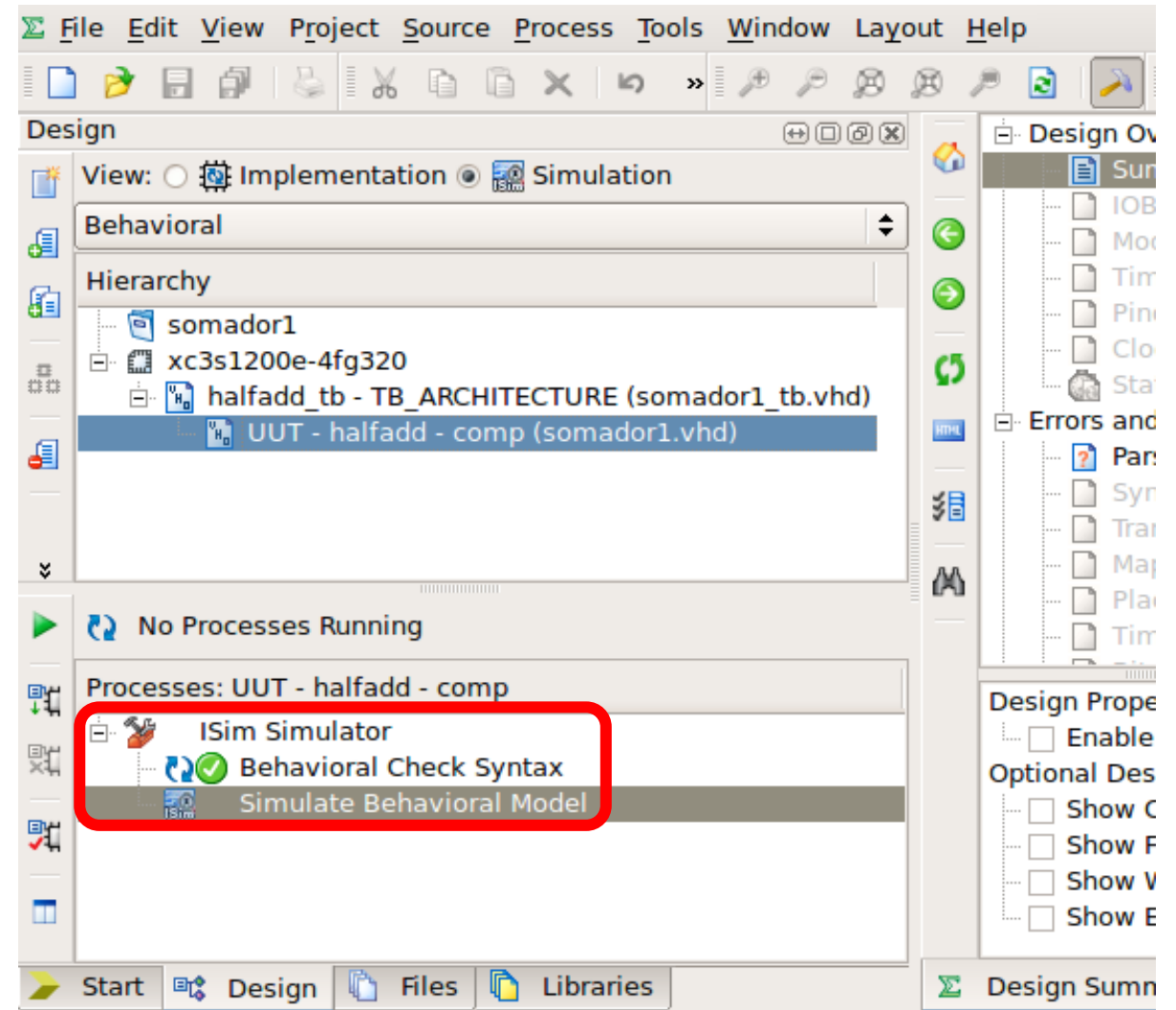
USANDO O SIMULADOR ISIM DA XILINX

8. Selecione o arquivo `halfadd_tb.vhd` na janela **Hierarchy**. A janela do ISE deve ficar como ao lado. Note que, depois desta seleção, na janela de processos (**Processes**) vê-se o acesso ao simulador



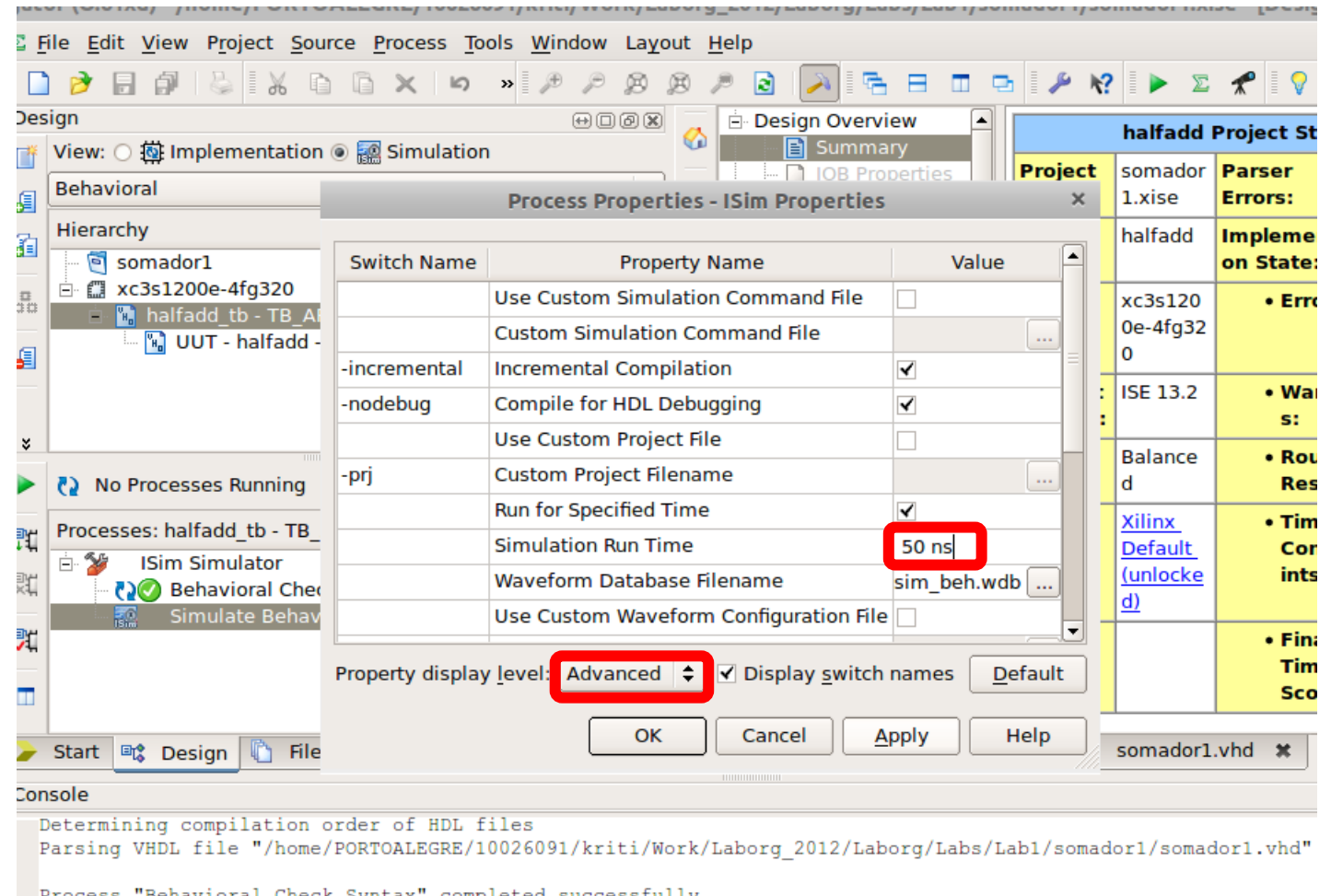
USANDO O SIMULADOR ISIM DA XILINX

9. Na janela **Processes**, dê duplo clique na opção **Behavioral Check Syntax**, para verificar que o testbench não possui erros de sintaxe
10. Abra a hierarquia de projeto, clicando no sinal **+** ao lado do arquivo **halfadd_tb.vhd** na janela **Hierarchy**. Selecione o arquivo **halfadd.vhd** e repita o processo de verificação sintática



USANDO O SIMULADOR ISIM DA XILINX

11. Para preparar a simulação, selecione de novo o arquivo `halfadd_tb.vhd` (janela [Hierarchy](#)), e clique com o botão direito do mouse na opção **Simulate Behavioral Model – Process Properties** (janela [Processes](#)). Na janela que se abrir, selecione (se já não estiver selecionada) a opção **Advanced** no item [Property Display Level](#) e mude o [Simulation Run Time](#) para **50ns**

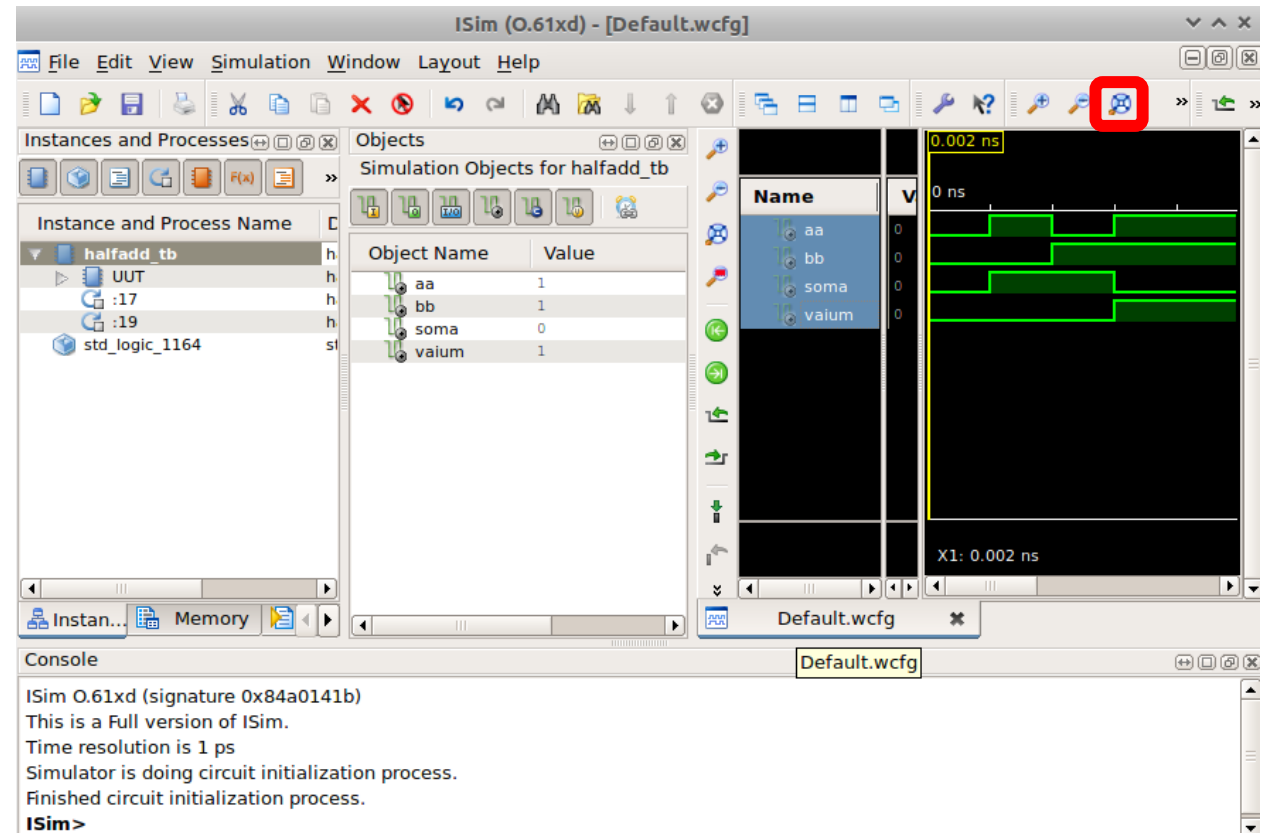


USANDO O SIMULADOR ISIM DA XILINX

12. Para simular, basta **fechar a janela Process Properties** clicando em **OK** e dar um **duplo clique** na opção **Simulate Behavioral Model** da janela **Processes**. Surge uma nova janela similar à de **fundo preto** com **linhas verdes** do desenho ao lado. Esta é a **janela de formas de onda** (em inglês, *waveform*) mostra os 50ns de simulação do circuito. Experimente usar os diversos botões disponíveis no simulador

Note: Esta forma de onda mostra todos os casos da tabela verdade do somador (duas variáveis, 4 casos)

Obs.: Lembrem-se de simular o **testbench** e não o projeto, selecionando o arquivo no topo da hierarquia. Para ver toda a onda na janela clique no botão “Zoom to Full View”



ADENDOS DO TUTORIAL 1

ADENDOS DO TUTORIAL 1

- Os **ADENDOS DO TUTORIAL 1** consistem em executar a captura e simulação de 2 projetos no ISIM, o simulador do ambiente ISE da Xilinx (o Projeto 2 e o Projeto 3). O Tutorial 1 em si pode também ser denominado Projeto 1. Nele viu-se
 - ✓ O aprendizado do uso do ambiente de simulação, pelo desenvolvimento de um meio-somador de 1 bit (`halfadd`)
 - ✓ Projeto 2
 - Usando as mesmas técnicas aplicadas ao Projeto 1, realizar o desenvolvimento de um somador de 4 bits sem vai-um (`full_adder`) e simular o mesmo
 - ✓ Projeto 3
 - Usando as mesmas técnicas aplicadas ao Projeto 1, realizar o desenvolvimento de um flip-flop mestre-escravo assíncrono com três arquiteturas distintas funcionalmente equivalentes (`ffd`)

PROJETO 2

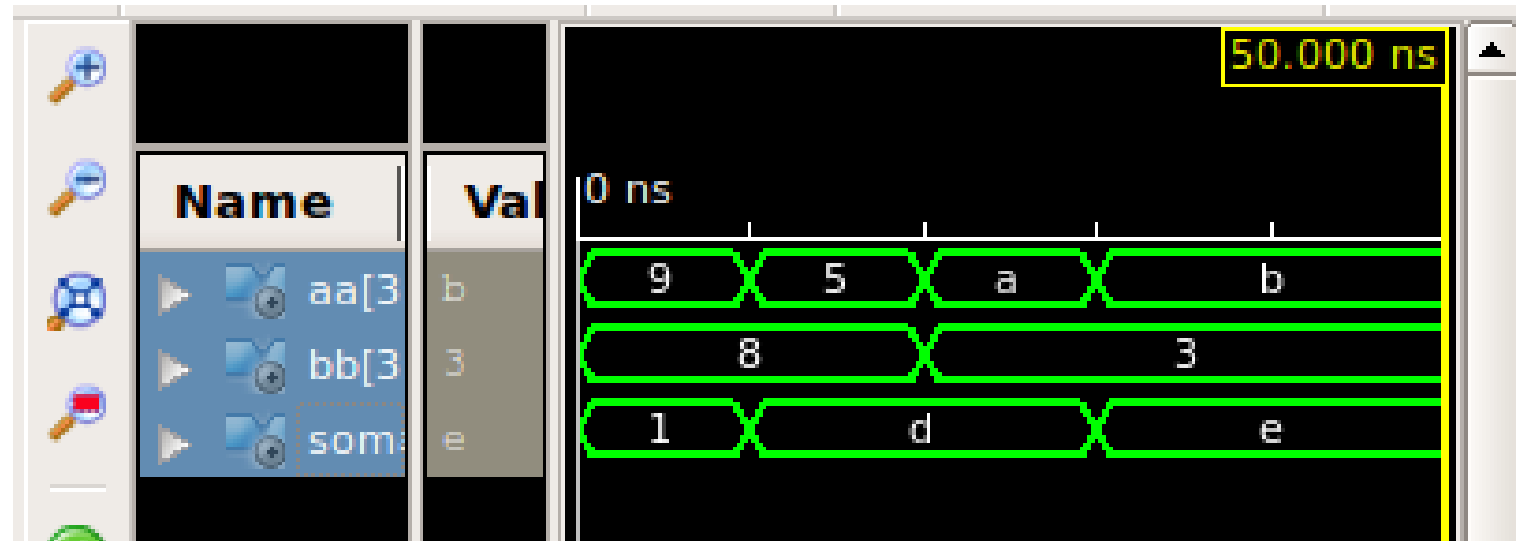
- Escreva um novo código (faça um novo projeto VHDL. Use a opção *Copy Project* do ISE) utilizando sinais não de 1 bit mas de 4 bits (`std_logic_vector(3 downto 0)`). O efeito é transformar o meio somador de 1 bit em um somador de 4 bits.
 - ✓ `std_logic` → fio e `std_logic_vector` → vários fios (barramento)
- A nova entidade VHDL deve parecer como o código abaixo

```
entity soma4b is
port (A, B: in std_logic_vector(3 downto 0);
      Soma: out std_logic_vector(3 downto 0));
end soma4b;
```
- O corpo da arquitetura VHDL deverá conter apenas `soma <= a + b;`
- **Observação importante:** VHDL é uma linguagem *case-insensitive*, e por isto a linha acima não produz erro, pois `soma <= a + b;` é absolutamente equivalente a `Soma <= A + B;`
- Deve-se incluir no cabeçalho do texto a linha: `use IEEE.std_logic_unsigned.all` (Porquê? Pesquisem o motivo...)
- Re-escreva o *testbench*, mudando o que se tem de mudar

PROJETO 2

- 4 bits podem ser representados como um único dígito em hexadecimal. No *testbench* pode-se escrever como abaixo (o prefixo x designa que o número está em hexa):
 - ✓ `aa <= x"9", x"5" after 10 ns, x"A" after 20 ns, x"B" after 30 ns;`
 - ✓ `bb <= x"8", x"3" after 20 ns;`

- Ao simular este circuito, aparece uma forma de onda similar à apresentada na Figura à direita. Os discentes devem testar outros valores no seu *testbench*



- O comportamento é o esperado? **Explique, analisando cada combinação!! Duvide!!**

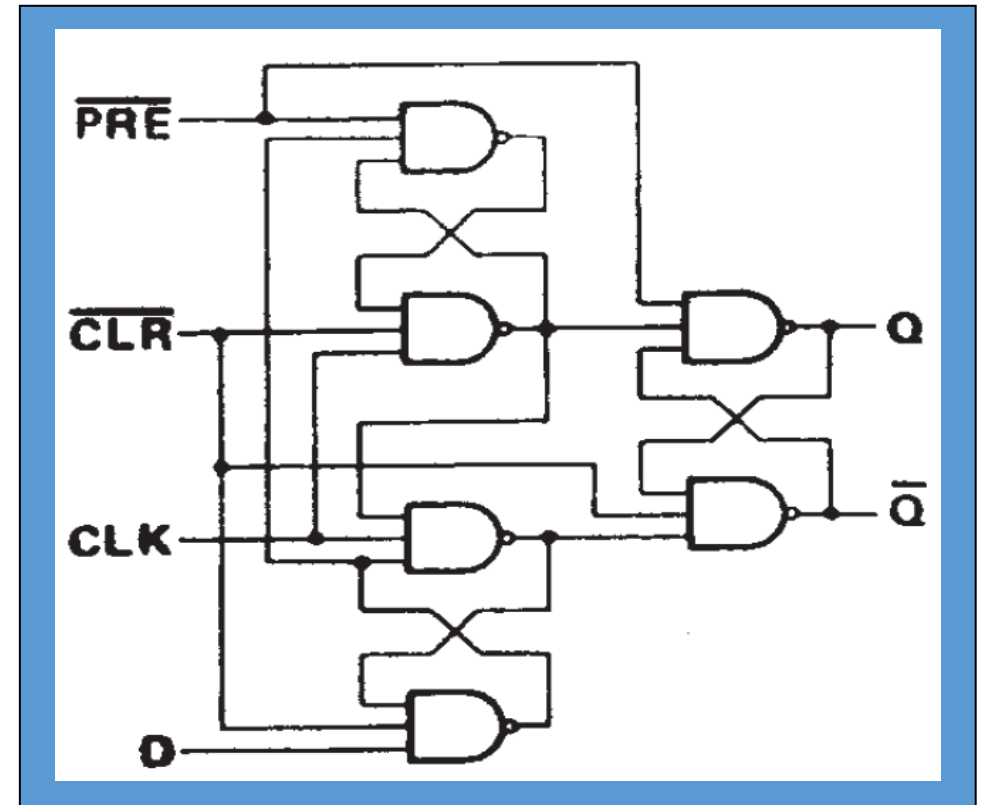
PROJETO 2

- Para gerar todas as combinações possíveis de entradas deste circuito seria necessário produzir 256 padrões (combinando os 16 valores distintos de A com os 16 valores distintos de B). Um trecho de código VHDL capaz de gerar estes 256 pares de estímulos é dado abaixo. **Estude-o e entenda-o!**

```
...
signal aa : std_logic_vector(3 downto 0) := "0000"; -- valor inicial
signal bb : std_logic_vector(3 downto 0) := "0000"; -- valor inicial
...
process (aa)
begin
    if (aa/=x"F") then
        aa <= aa+x"1" after 10ns; -- a cada 10ns
    else aa <= x"0" after 10ns; -- incrementa A
    end if;
end process;
process (bb)
begin
    if (bb/=x"F") then
        bb <= bb+x"1" after 160ns; -- a cada 16
    else bb <= x"0" after 160ns; -- valores de A,
    end if; -- incrementa B
end process;
```

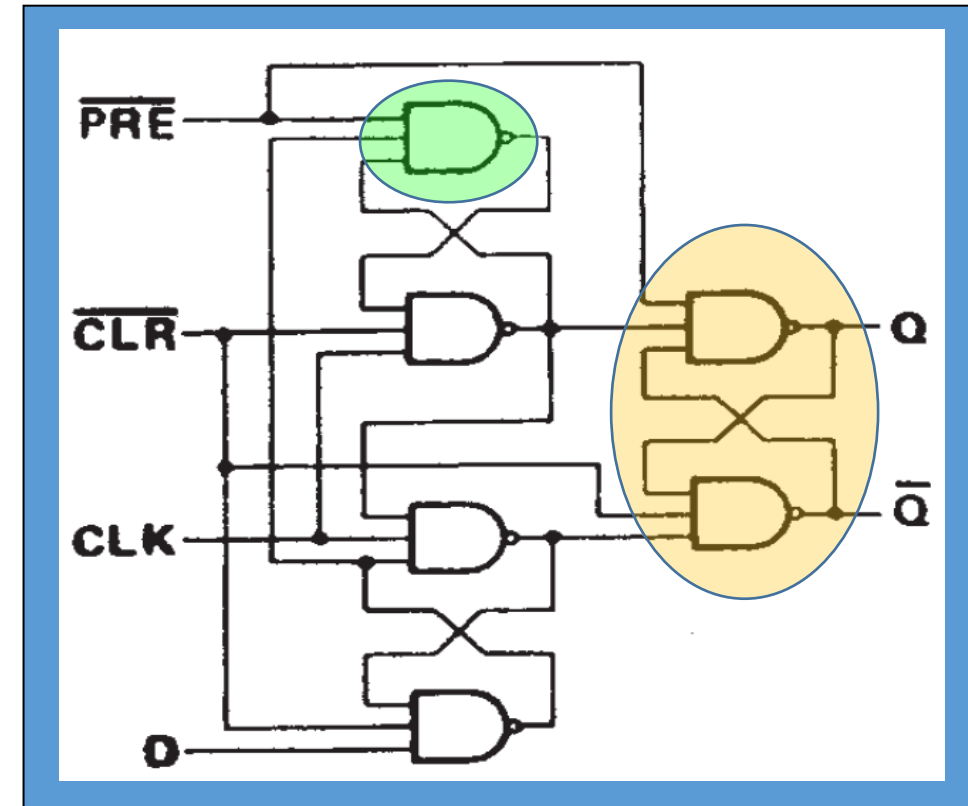
PROJETO 3

1. Implemente o circuito de seis portas lógicas ao lado em VHDL – aqui **é proibido usar o comando *process* de VHDL!** Este comando será tema de aula ulterior ...
2. Crie um projeto com o ISE que vai conter a entidade (***entity***) correspondente a este circuito e as três arquiteturas (***architecture***) descritas mais adiante
3. Crie um ***testbench*** para cada versão (***architecture***) do circuito. Este deve instanciar cada circuito, gerar estímulos para todas as suas 4 entradas e aplicar os estímulos à instância da arquitetura em questão
4. Simule cada ***testbench*** e procure descrever textualmente como o circuito funciona
5. **Importante: A descrição VHDL deve refletir exatamente a estrutura do diagrama de portas ao lado, não apenas seu comportamento! Por isto estabelece-se aqui a proibição de usar *process*, que descreve tipicamente o comportamento de um circuito, não sua estrutura**

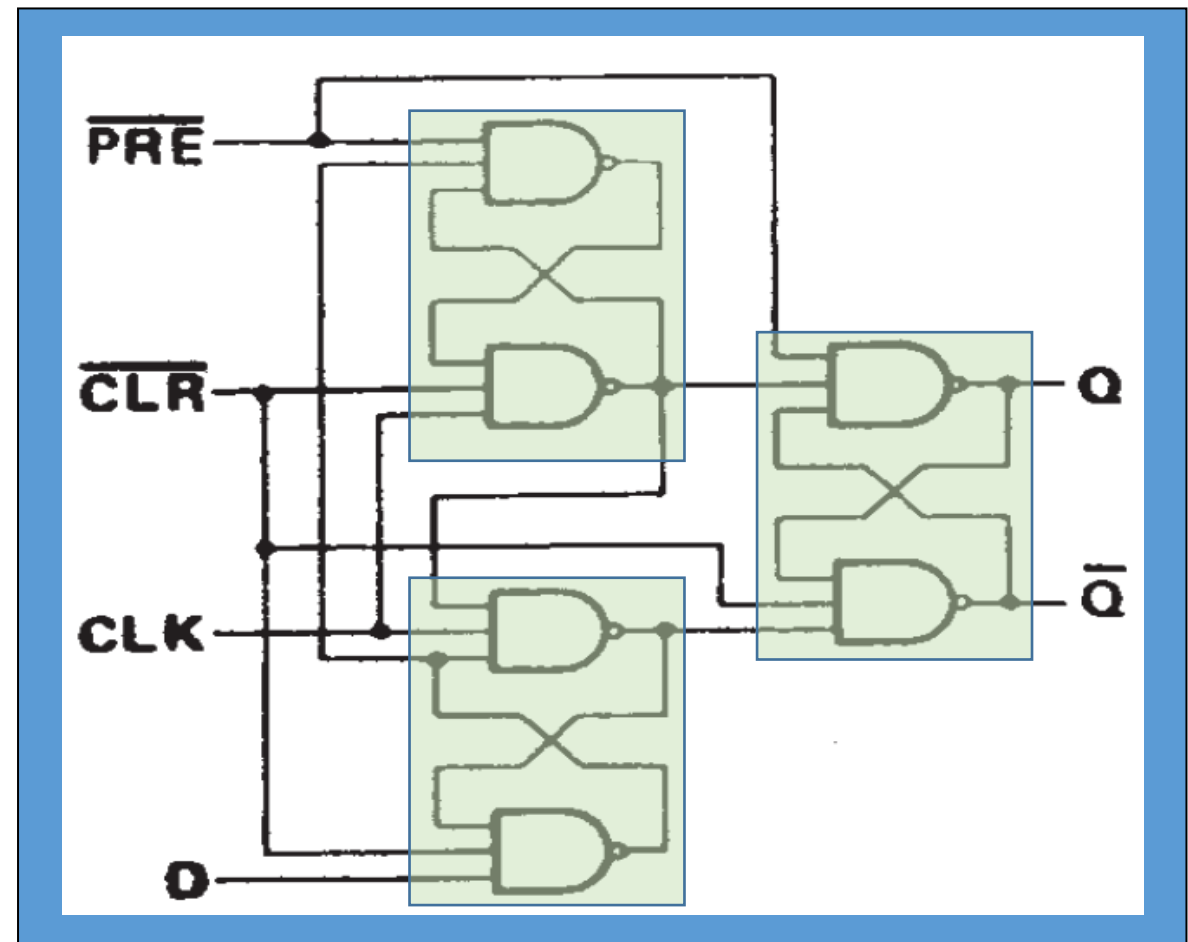
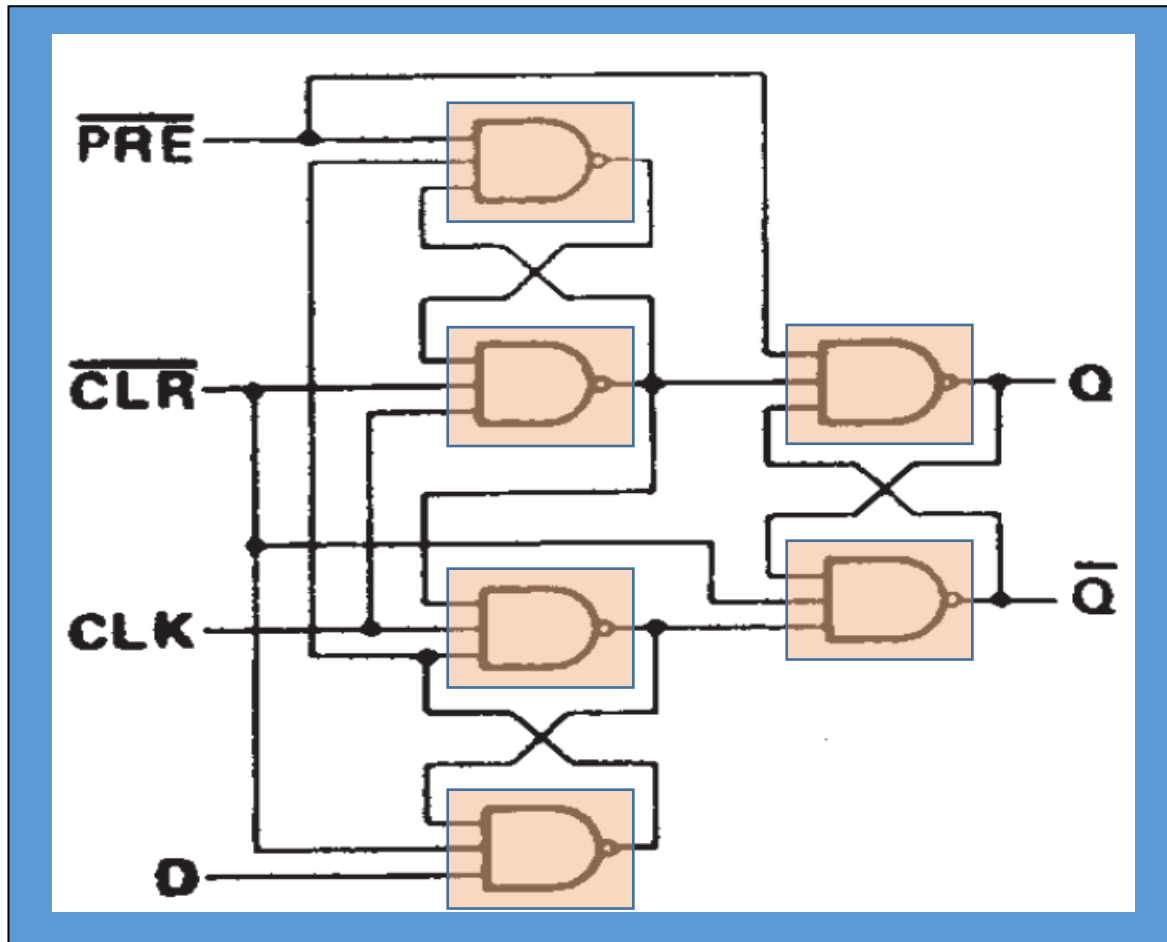


PROJETO 3 – IMPLEMENTAÇÕES ALTERNATIVAS

1. Observando o diagrama de portas original, nota-se que existem estruturas similares entre si no desenho
 - O circuito só usa portas NAND de 3 entradas (NAND3i)
 - Há três pares de NAND3i interconectados do mesmo jeito (saída de uma NAND é entrada da outra e vice-versa, formando um *latch*)
2. Com base nas observações acima crie duas novas versões deste circuito em VHDL
 - Uma usando 6 vezes uma NAND3i como par entidade/arquitetura
 - Uma usando 3 vezes um *latch* com 4 pinos de entrada e 2 de saída como par entidade/arquitetura



PROJETO 3 – IMPLEMENTAÇÕES ALTERNATIVAS



Estruturas de implementação alternativas com E/A Nand3i e E/A latch_4inps

PROJETO 3

➤ Informações adicionais

- ✓ Trata-se de um **flip-flop tipo D sensível à borda de subida do relógio** (sinal **CLK**), com **sinais assíncronos** (isto é independentes e prioritários em relação ao sinal de relógio) de **preset** (sinal **$\overline{PRE}$**) e **reset** (sinal **$\overline{CLR}$**). De fato, este é o diagrama de portas de cada um dos FFs encapsulados em um circuito integrado [TTL 74LS74](#)
- ✓ As saídas são os sinais **Q** e **$\overline{Q}$** , tipicamente assumindo valores opostos, e mostram o valor do bit armazenado no **FF**
- ✓ O sinal **$\overline{PRE}$** em **0** força a saída **Q** para **1** (e **$\overline{Q}$** para **0**), enquanto **$\overline{CLR}$** em **0** força **Q** para **0** (e **$\overline{Q}$** para **1**). Eles não devem nunca ser **0** ao mesmo tempo. Qualquer um destes sinais em **0** faz com que as bordas do relógio sejam ignoradas (elimina o efeito destas)
- ✓ Seguindo a regra geral de circuitos síncronos, **nenhuma entrada deve mudar** ao mesmo tempo que o relógio. **Porquê??? Pense e pesquise os motivos...**
- ✓ Cada borda de subida de **CLK** sem **$\overline{PRE}$** nem **$\overline{CLR}$** ativados (ou seja, com ambos sinais em **1**) provoca a amostragem e o armazenamento do valor de **D**

PROJETO 3

- Notem que o circuito a implementar é assíncrono (devido aos laços de realimentação combinacionais), e cria um componente básico para projetos síncronos. Como já foi mencionado, trata-se de um flip-flop D sensível à borda de subida do clock
- Cuidados a tomar
 - ✓ Ao fazer o *testbench*, respeitem a relação de temporização entre sinais de um circuito síncrono
 - Quando o clock mudar na sua borda sensível ($0 \rightarrow 1$, borda de subida), **NENHUM** sinal amostrado por ele (*D*, neste caso) pode mudar ao mesmo tempo! Lembrem-se dos conceitos de tempo de *setup* e *hold*! **Se lembram???**
 - Os sinais $\overline{\text{CLR}}$ e $\overline{\text{PRE}}$ também devem respeitar esta regra!
 - ✓ Notem, nomes de sinais no *testbench* podem ser idênticos aos nomes dos pinos dos objetos que o *testbench* instancia

RESUMO DO TUTORIAL 1

- O **Tutorial 1** compreende ao todo 3 projetos ISE
 - ✓ Meio-somador de 1 bit ([half_adder](#))
 - ✓ Somador de 4 bits sem vai-um ([full_adder](#))
 - ✓ Flip-flop D sensível à borda de subida do clock ([ffd](#)) – com três arquiteturas alternativas
- O que deve ser gerado ao final da realização do Tutorial 1 (Será disponibilizada a parte de implementação do tutorial, o relatório cabe aos discentes elaborar, para uso próprio)
 - ✓ Geram um material (para uso próprio), contendo os seguintes **arquivos**
 - Um relatório em PDF com formas de onda, descrevendo como cada circuito funciona e sua implementação de acordo com o modelo apresentado (guardem para uso futuro)
 - Para cada 1 dos 3 projetos descritos acima, o [código VHDL de cada implementação \(3 no caso do Projeto 3\)](#) e o [testbench que valida a implementação](#)
 - Por exemplo
 - [half_adder/half.vhd](#)
 - [half_adder/tb_half.vhd](#)
 - [full_adder/full.vhd](#)
 - ...

TUTORIAL 2: A PROTOTIPAÇÃO de HARDWARE EM FPGAS XILINX

O ARQUIVO A PROTOTIPAR

1. Abaixo aparece um circuito somador, que deve ser prototipado pelos alunos (compare com a implementação do Projeto 1 e do Projeto 2 do Tutorial 1). Use este arquivo como base neste tutorial

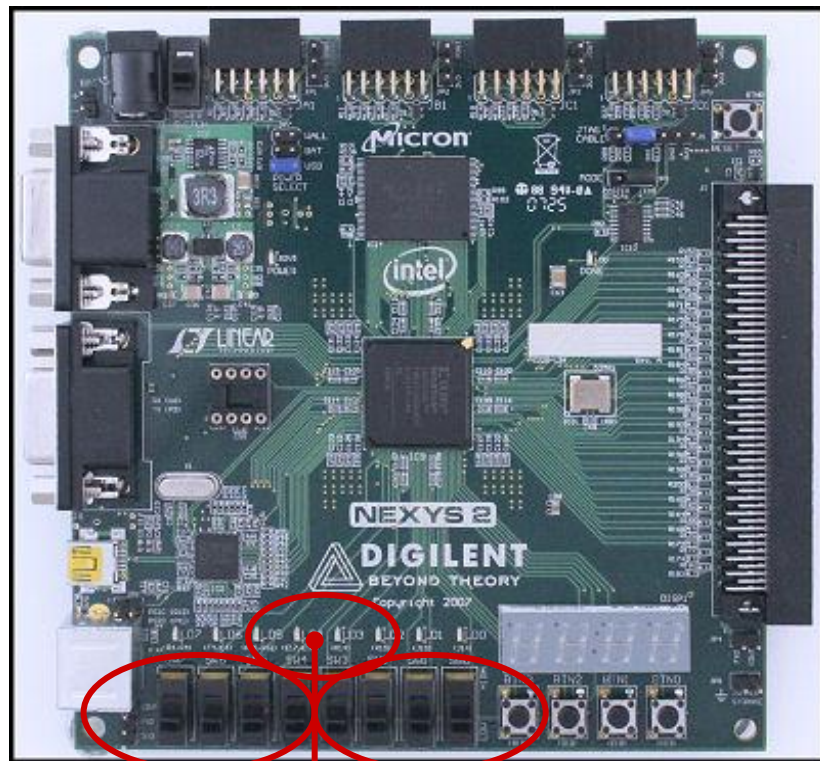
```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all; --Para permitir soma de std_logic

entity soma4b is
port (A, B: in std_logic_vector(3 downto 0);
      Soma: out std_logic_vector(3 downto 0));
end soma4b;

architecture soma4b of soma4b is
begin
    Soma <= A + B; --Soma de dois vetores de 4 bits
end soma4b;
```

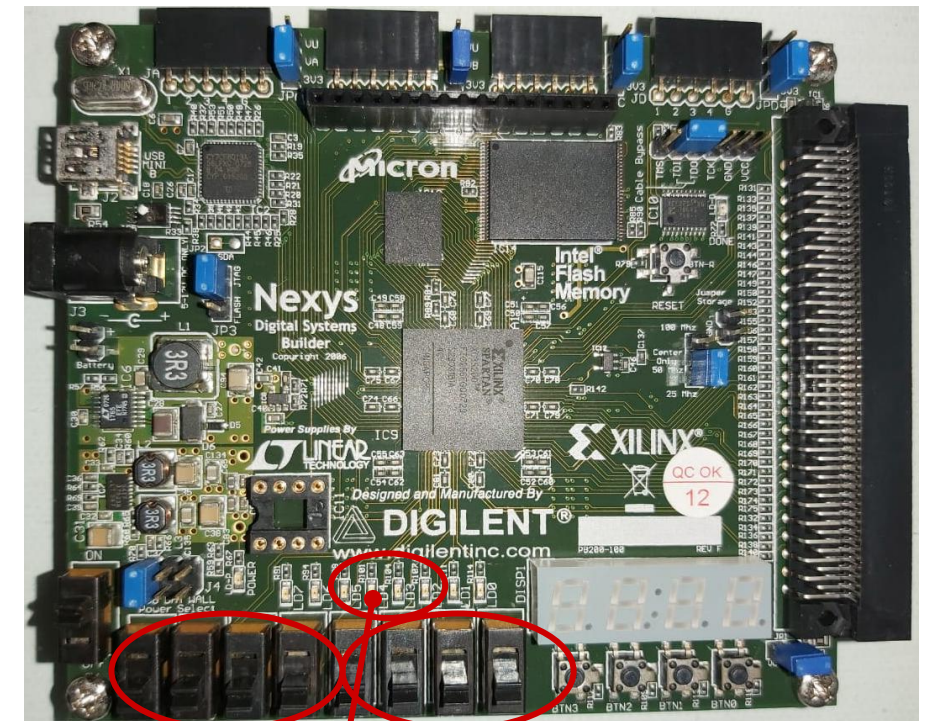
ONDE AS ENTRADAS E SAÍDAS SE CONECTAM?

- Placas de prototipação têm recursos de entrada e saída
 - ✓ LEDs, chaves, displays, conectores (serial, USB, Ethernet...)



A(3 downto 0); B(3 downto 0);
Soma(3 downto 0)

- ✓ As Imagens aqui mostram um exemplo de atribuição de dispositivos de entrada e saída das placa Nexys 2 e Nexys para prototipar o somador de 4 bits

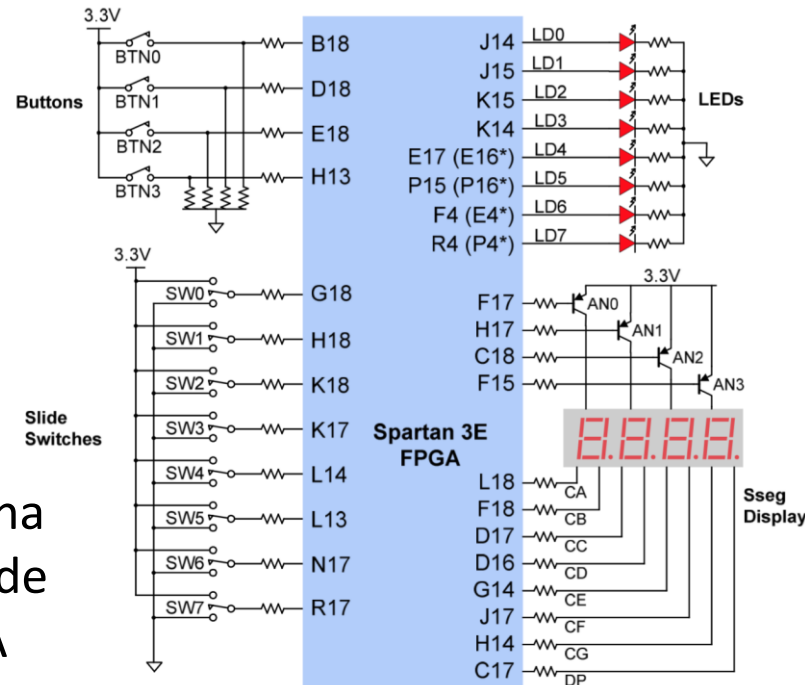


A(3 downto 0); B(3 downto 0);
Soma(3 downto 0)

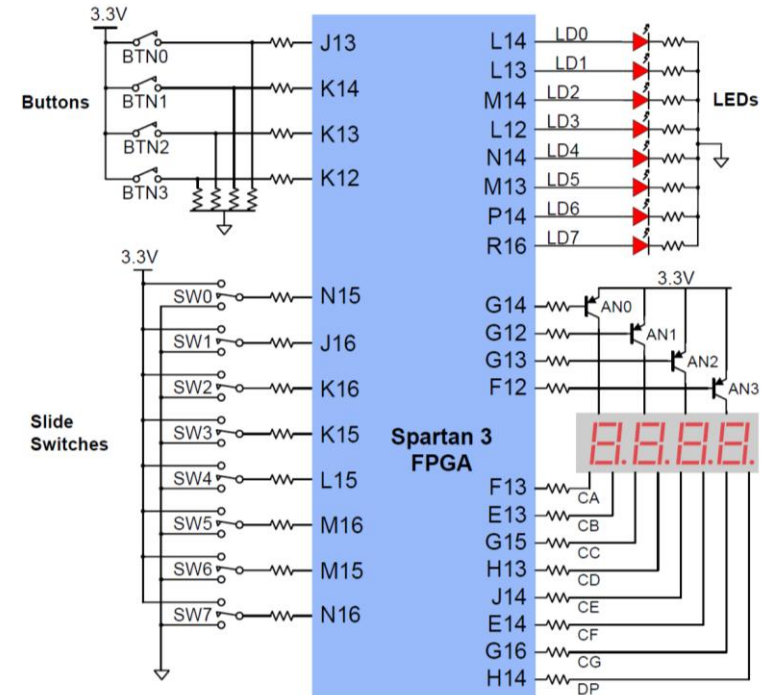
ONDE AS ENTRADAS E SAÍDAS SE CONECTAM?

2. Abra o manual de referência da placa em questão no material de apoio do Moodle ou aqui ([Nexys2_rm.pdf](#) ou [Nexys_rm.pdf](#)). Achar no manual uma das Figuras ao lado

✓ Um arquivo de projeto relaciona as entradas e saídas da entidade VHDL topo com pinos do FPGA conectados aos recursos da placa



Nexys 2*



Nexys

- ✓ Este arquivo, para os FPGAs Xilinx mais antigos como Spartan3 ou Spartan3E se denomina **UCF**, uma abreviatura de **User Constraints File** (ou seja, Arquivo de Restrições [impostas pelo] do Usuário)
- ✓ **Atenção: Para a Nexys2 usando o dispositivo XC3S500E use os pinos sem * ao lado do código. Para as Nexys 2 com o dispositivo XC3S1200E use os pinos com * ao lado do código (para LD4-LD7)**

COMO AS ENTRADAS E SAÍDAS SE CONECTAM?

3. Crie o arquivo UCF – Seu conteúdo define a relação entre cada nome de fio/pino em VHDL ao pino físico a associar a este no FPGA, abaixo é mostrada uma sintaxe válida

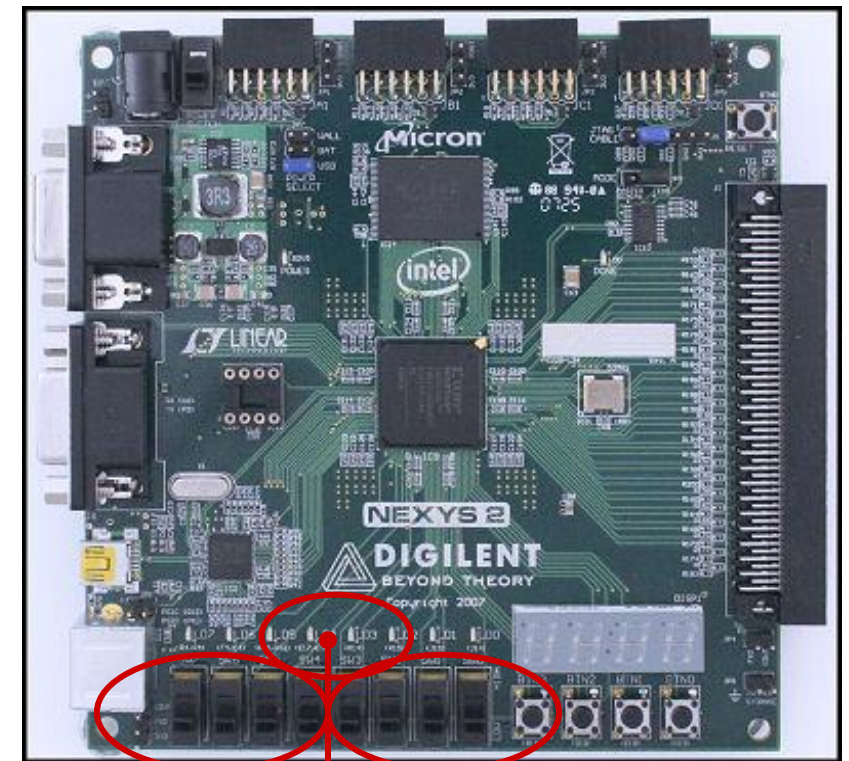
```
### UCF do Projeto Somador de 4 Bits para a Nexys2 com XC3S1200E
### Adaptar a pinagem para Nexys2 com XC3S500E ou para Nexys
```

```
NET "A[3]" LOC = R17; # Bit 3 do vetor A
NET "A[2]" LOC = N17; # Bit 2 do vetor A
NET "A[1]" LOC = L13; # Bit 1 do vetor A
NET "A[0]" LOC = L14; # Bit 0 do vetor A
```

```
NET "B[3]" LOC = K17; # Bit 3 do vetor B
NET "B[2]" LOC = K18; # Bit 2 do vetor B
NET "B[1]" LOC = H18; # Bit 1 do vetor B
NET "B[0]" LOC = G18; # Bit 0 do vetor B
```

```
NET "Soma[3]" LOC = P16;
NET "Soma[2]" LOC = E16;
NET "Soma[1]" LOC = K14;
NET "Soma[0]" LOC = K15;
```

- O arquivo UCF ao lado vale para a placa Nexys 2 com XC3S1200E
- Para a placa Nexys com XC3S500E ou para Nexys, altere a pinagem conforme imagens do slide anterior

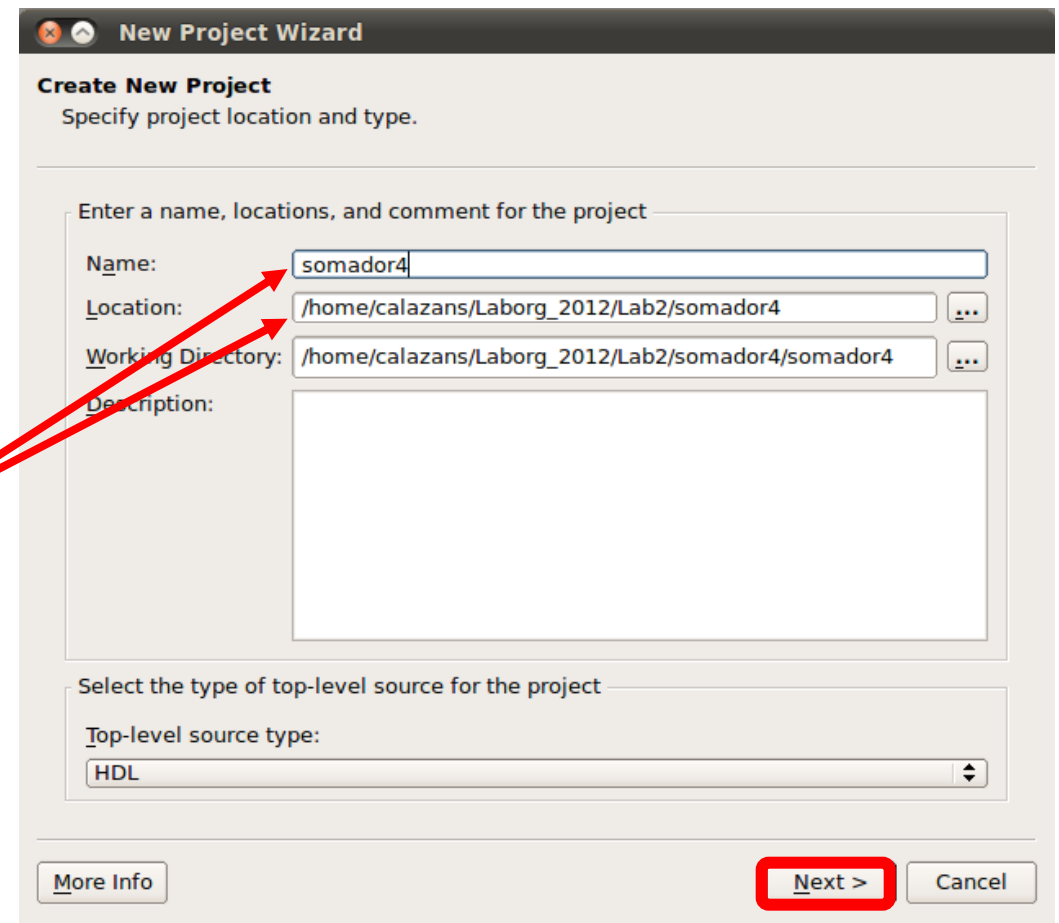


A(3 downto 0); B(3 downto 0);
Soma(3 downto 0)

AMBIENTE DE SÍNTESE: ISE

4. Crie um diretório, colocando neste os arquivos VHDL ([somador4.vhd](#)) e o arquivo UCF ([somador4.ucf](#))
5. Abra a ferramenta ISE como descrito nas transparências do Tutorial anterior e crie um novo projeto ([File](#) → [New Project](#)), como ao lado

Cuidado: Não podem haver espaços em branco ou caracteres especiais, nem no nome do caminho para o projeto, nem no nome do projeto



DEFINIÇÃO DO FPGA DA PLACA DE PROTOTIPAÇÃO

6. Para a placa que estamos trabalhando, o FPGA é um dispositivo da família Spartan3E (Nexys 2) ou Spartan3 (Nexys). Faça as escolhas na janela, como mostrado ao lado

**Características do dispositivo FPGA,
para a placa Nexys 2
(para o dispositivo 500, trocar XC3S1200E
por XC3S500E, o resto é igual)**

Para a placa Nexys, usem

- **Family: Spartan3**
- **Device: XC3S200**
- **Package: FT256**
- **Speed: -4**

New Project Wizard

Project Settings
Specify device and project properties.

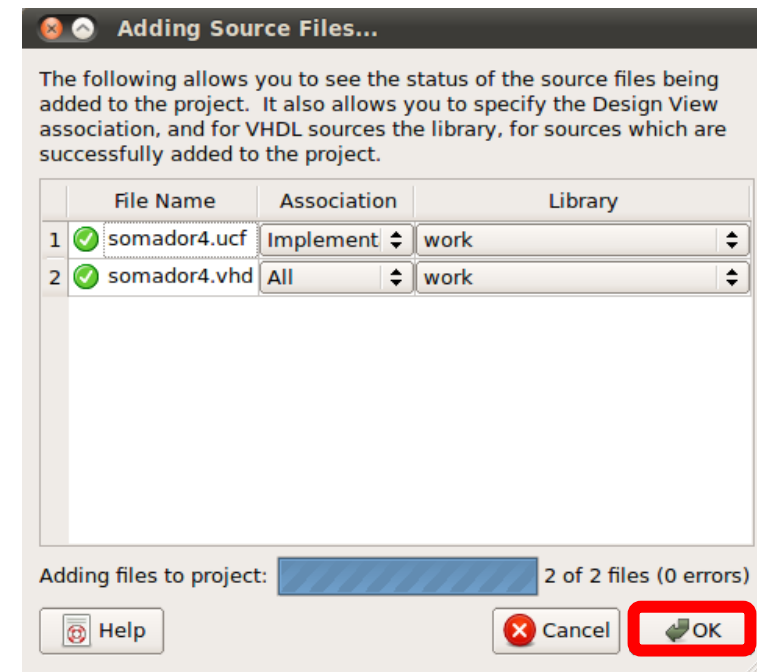
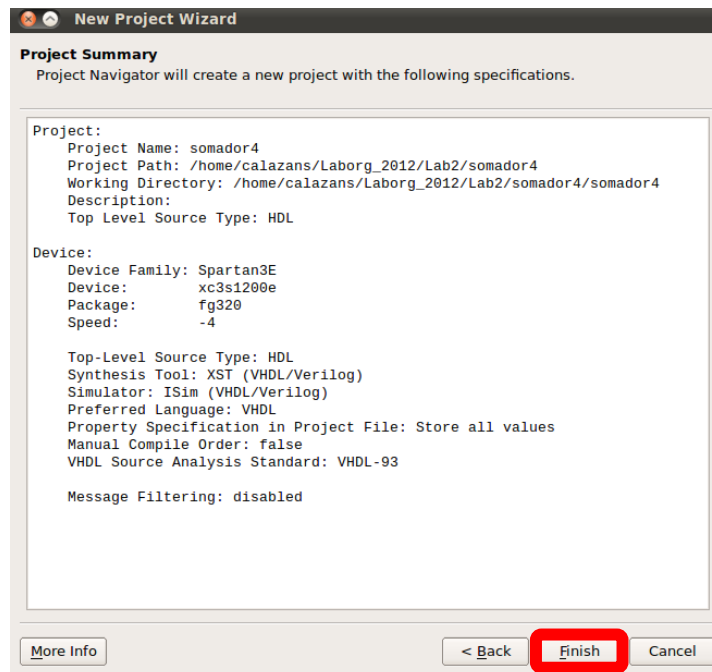
Select the device and design flow for the project

Property Name	Value
Product Category	All
Family	Spartan3E
Device	XC3S1200E
Package	FG320
Speed	-4
Top-Level Source Type	HDL
Synthesis Tool	XST (VHDL/Verilog)
Simulator	ISim (VHDL/Verilog)
Preferred Language	VHDL
Property Specification in Project File	Store all values
Manual Compile Order	☐
VHDL Source Analysis Standard	VHDL-93
Enable Message Filtering	☐

[More Info](#) [< Back](#) [Next >](#) [Cancel](#)

DEFINIÇÃO DO FPGA DA PLACA DE PROTOTIPAÇÃO

7. A próxima janela é só um resumo do projeto, clique no botão **Finish**, como marcado abaixo na janela esquerda
8. A seguir, na janela principal, aparece o projeto vazio. Deve-se então acrescentar os arquivos fonte (**somador4.vhd** e **somador4.ucf**) gerados anteriormente. Na janela **Hierarchy**, clicar com o botão direito no ícone do dispositivo (**xc3s1200e-4fg320/xc3s200-4ft256**). No menu que surge, escolha **Add Copy of Source**. Procure no disco e adicione os dois arquivos ao projeto. Um resultado possível é mostrado abaixo, à direita



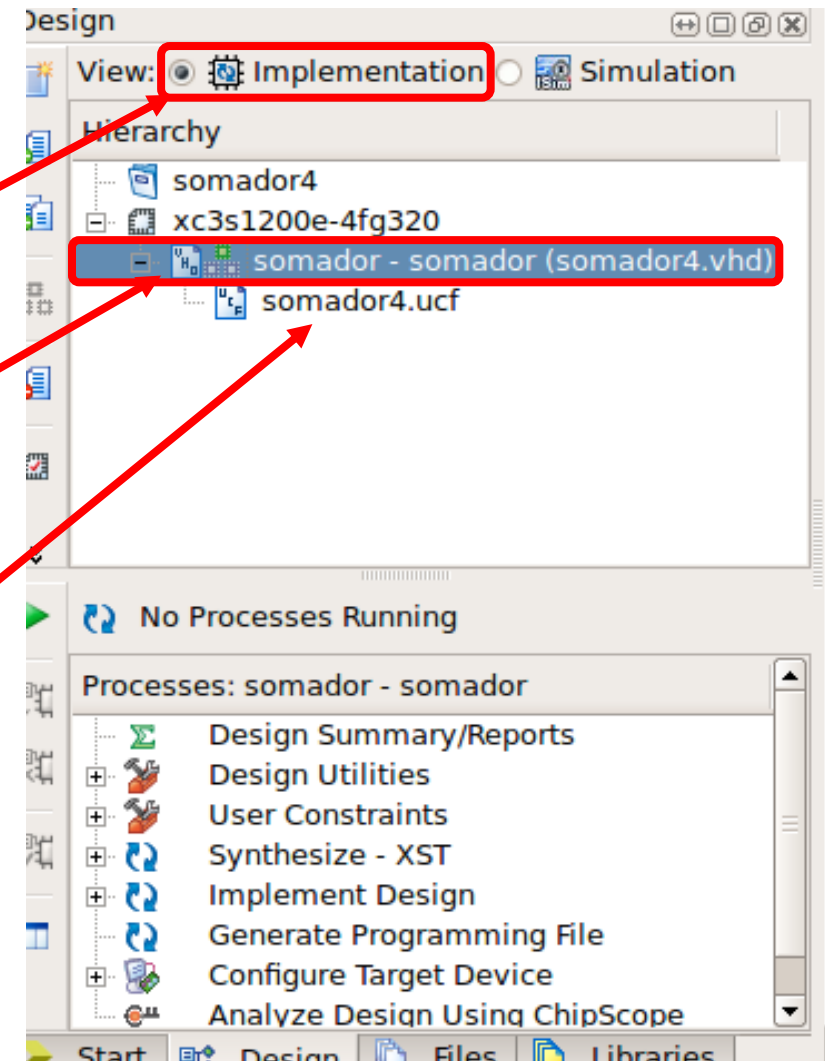
AMBIENTE ISE – BROWSER DO PROJETO

- Se todos os passos de criação foram corretamente seguidos, deve-se ter algo similar à imagem ao lado

Lembrem-se: aqui trabalha-se com síntese (implementação) e não com simulação

Arquivo do topo da hierarquia do projeto, ou seja, o circuito a sintetizar

O arquivo UCF



PASSO 1 DA SÍNTESE: SÍNTESE LÓGICA

9. O processo de síntese transforma o VHDL em estruturas que depois serão mapeadas para o FPGA. Para executar a síntese lógica, você deve dar um duplo click em “[Synthesize XST](#)” na janela de Processos ([Processes](#))
- ✓ Ao final da síntese tem-se o relatório de ocupação, ver exemplo de resultado abaixo

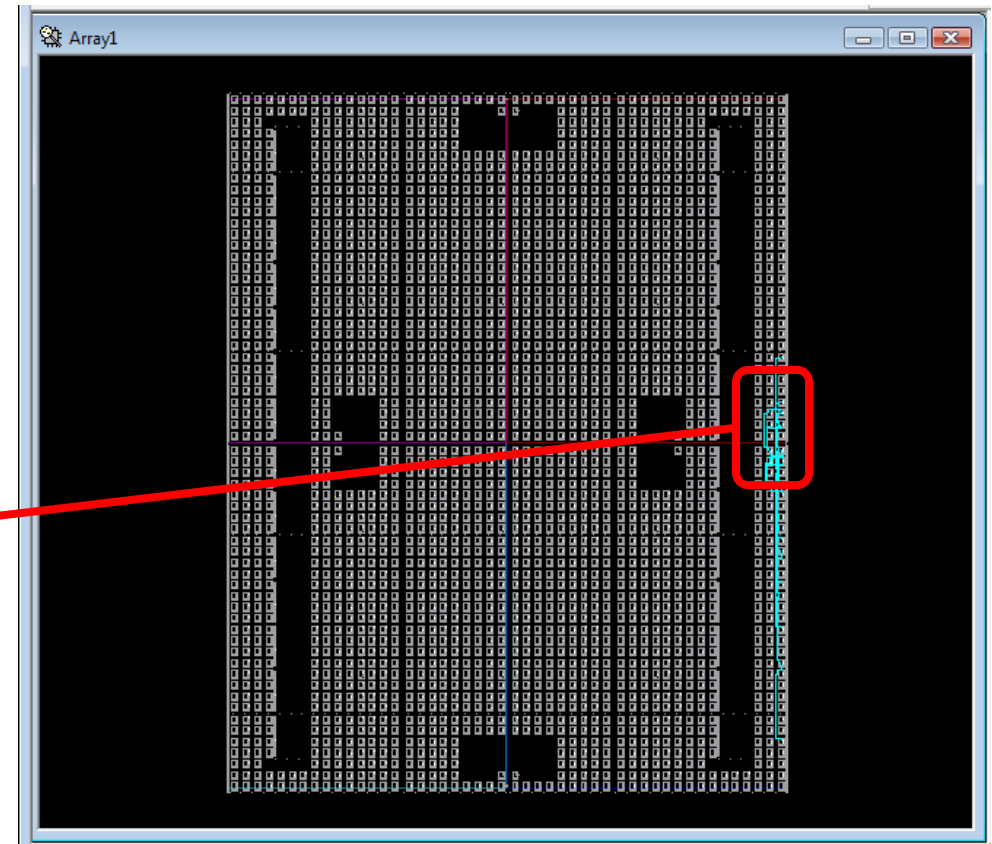
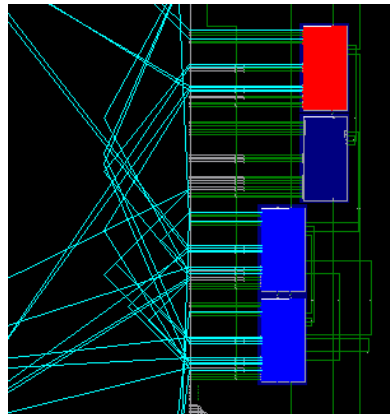
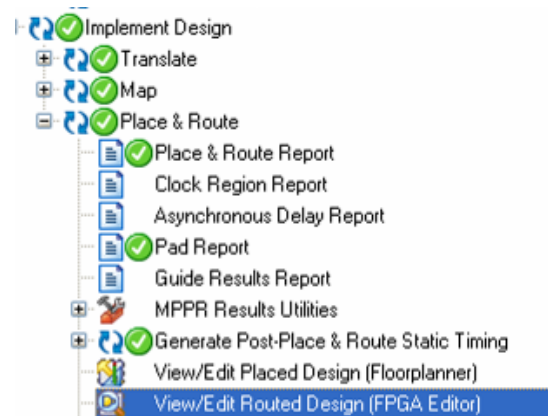
Device Utilization Summary (estimated values)				[-]
Logic Utilization	Used	Available	Utilization	
Number of Slices	3	8672	0%	
Number of 4 input LUTs	6	17344	0%	
Number of bonded IOBs	12	250	4%	

6 LUTs usadas, de 17.344 disponíveis
(Nexys 2 – XC3S1200E)

12 pinos usados, de 250 disponíveis
(Nexys 2 – XC3S1200E)

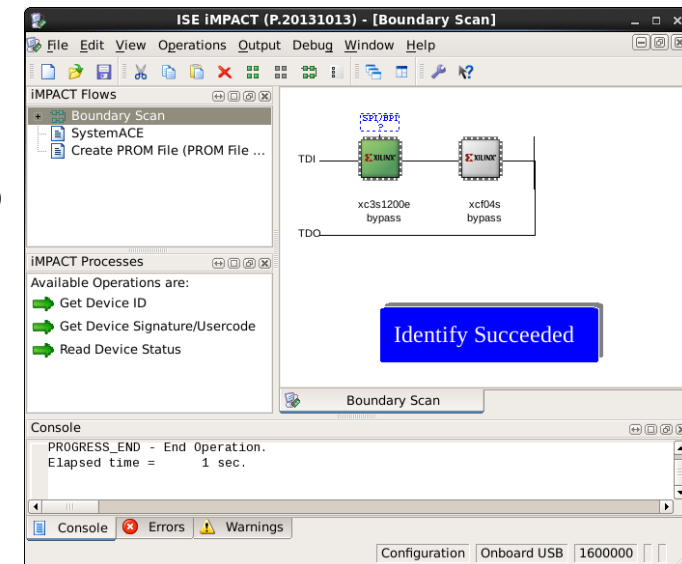
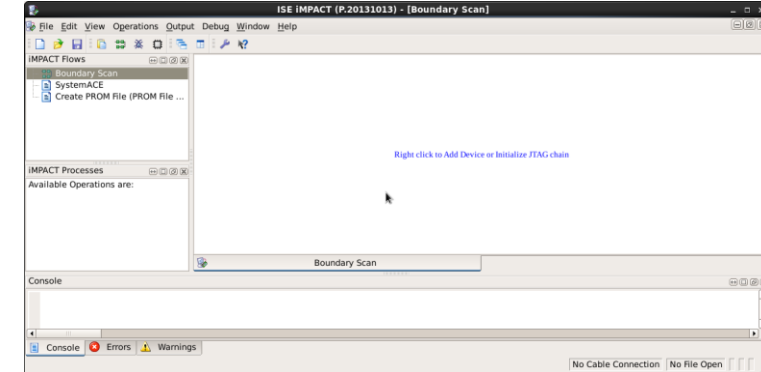
PASSO 2 DA SÍNTESE: SÍNTESE FÍSICA

9. Deve-se fazer o posicionamento e o traçado de conexões dentro do FPGA. Para isso, dê um duplo click em “**Implement Design**” na janela **Processes**
10. Em seguida, dê um duplo click em “**Generate Programming File**” na janela **Processes**
11. (Opcional) Selecione FPGA Editor (**Obs: isto só funciona no Windows, esta não existe em todo SO Linux**), execute o programa e visualize o “**layout**” gerado automaticamente pelo processo de síntese



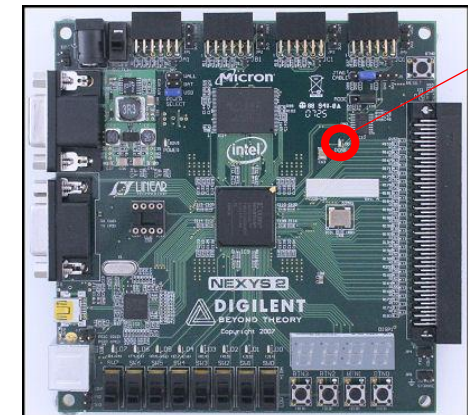
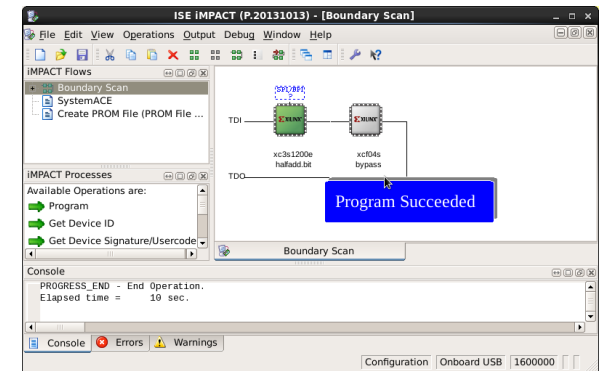
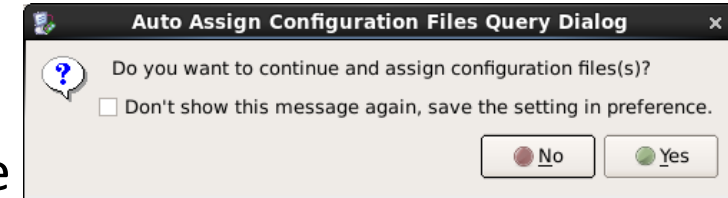
CONFIGURAR O FPGA – VIA ISE

12. Conecte a sua placa a um conector USB da sua estação de trabalho, garantindo que ela esteja ligada (led vermelho POWER, à direita do conector VGA, aceso)
13. No ISE, na **View: Implementation**, selecione (clique) o topo da hierarquia de projeto na janela **Hierarchy** o arquivo **halfadd.vhd**
14. Na janela **Processes** ative (dê duplo click) na opção **Configure Target Device**, e aceite o possível Alerta (**Warning**) que surgir. Isto deve abrir a janela do aplicativo **Impact** de configuração do FPGA
15. Na janela **Impact Flows**, dê um duplo click na opção **Boundary Scan**. Isto deve abrir a janela **Boundary Scan** no painel maior à direita no Impact.
16. Clique com o botão direito do mouse na janela **Boundary Scan**. No menu que surge escolha a terceira opção, **Initialize Chain**. Isto deve desenhar a cadeia JTAG na janela, que começa no pino **TDI** do FPGA, sai deste e vai para o chip de memória de configuração e sai pelo pino **TDO** deste



CONFIGURAR O FPGA – VIA ISE

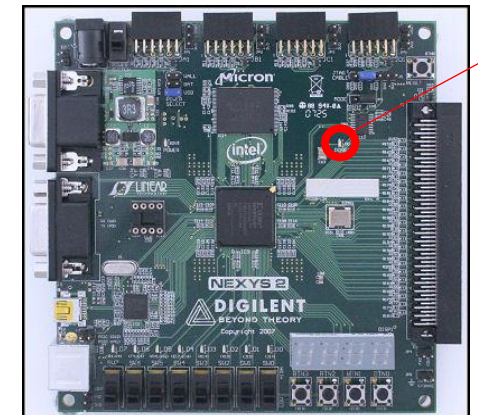
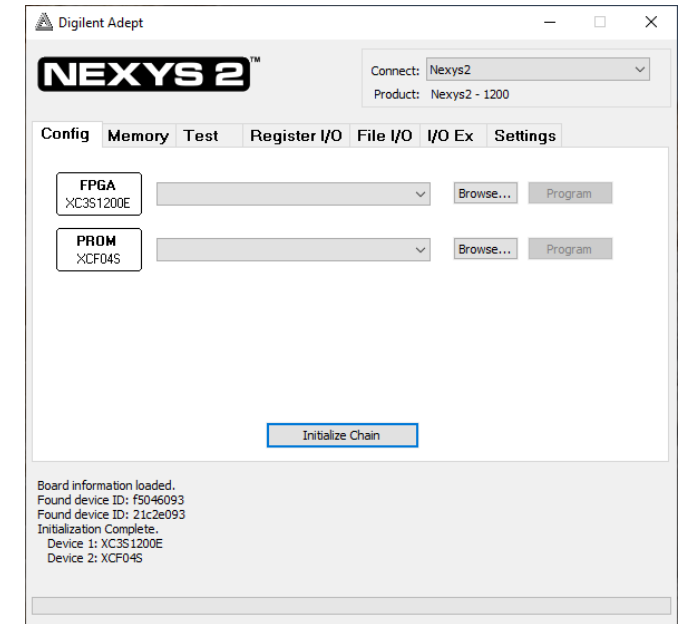
17. Depois da identificação da cadeia de JTAG, surge uma caixa de pergunta, responda **Yes**. Surge uma janela que permite navegar no sistema de arquivos. Se já não estiver lá, vá até a raiz do seu projeto e escolha o arquivo de configuração, **halfadd.bit**. Aceite a vinculação dele ao FPGA
18. Em seguida surge uma outra caixa de pergunta, responda **No** (Não vamos usar a memória de configuração). Na janela de escolha que aparece selecione **Bypass**. Na próxima janela que aparece, escolha OK.
19. Clique sobre o FPGA (símbolo deve estar pintado de **verde**) com o botão direito do mouse e selecione a primeira opção (Program) no menu que surge. Isto deve disparar o processo de configuração do FPGA.
20. Após a configuração terminar, se tudo deu certo acende-se o led **amarelo** da placa, posicionado sobre o texto que lhe dá nome, **DONE**



Led
DONE

ALTERNATIVA: CONFIGURAR O FPGA – VIA SW ADEPT DA DIGILENT

12. Conecte a sua placa a um conector USB da sua estação de trabalho
13. Instale e execute o software Adept da Digilent que pode ser obtido em <https://store.digilentinc.com/digilent-adept-2-download-only/>. A imagem ao lado mostra a tela inicial do software
14. Clique no botão “Initialize Chain” o que deve identificar os dois dispositivos programáveis da placa (para a Nexys 2 dos laboratórios estes serão o FPGA XC3S1200E e a memória XCF04S)
15. Para configurar o FPGA aperte o botão “Browse” na mesma linha que o símbolo do FPGA e ache o arquivo de configuração (denominado, por exemplo, `halfadd.bit`), selecionando-o
16. Aperte o botão “Program”, que deve disparar o processo de configuração do FPGA.
17. Após a configuração terminar, se tudo deu certo acende-se o led amarelo da placa, posicionado sobre o texto que lhe dá nome, **DONE**



ADENDOS DO TUTORIAL 2

PROJETO 1

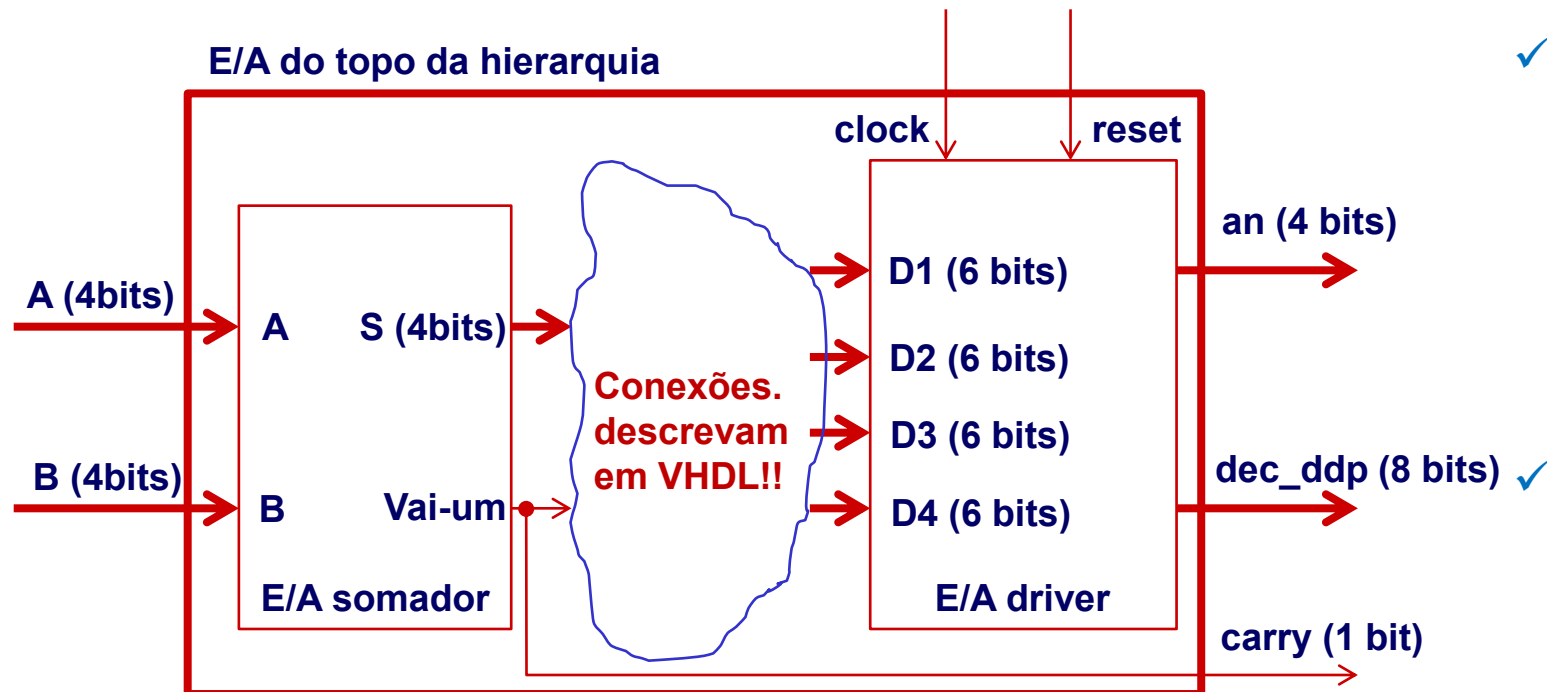
- **Projeto 1** - consiste em acrescentar o cálculo de “vai-um” no circuito do somador apresentado no “Tutorial de Prototipação em FPGA” desta apresentação. Segue uma sugestão de modificações no código para resolver o problema
 - ✓ Modificar a saída “Soma” para ser um vetor de 5 bits.
 - ✓ Declarar dois sinais internos ‘AA’ e ‘BB’, ambos de 5 bits
 - ✓ Fazer a “Soma <= AA + BB”, e criar ‘AA’ e ‘BB’ através de uma concatenação com ‘0’, e.g.:
`AA <= '0' & A; -- o símbolo & significa concatenação em VHDL`
- Modificar o UCF para que um dos LEDs não usados (por exemplo o LD7) seja associado ao “vai-um” gerado (o quinto bit da soma)
- Simular o VHDL com o mesmo testbench do [Tutorial 1](#) para este novo VHDL
- Aplicar os passos descritos na seção “Um Tutorial de Prototipação em FPGAs Xilinx” ao novo VHDL gerado, prototipando o novo hardware

PROJETO 2

- **Projeto 2** – fazer o resultado da soma aparecer nos mostradores (*displays*) de 7 segmentos. Segue uma sugestão de modificação do projeto para tanto. Use como base o **Projeto 1** e suponha que o resultado da soma vá para o mostrador mais à esquerda da placa, e que o “vai-um” seja associado a algum LED (pode ser um LED não usado, como LD4 a LD7 ou um ponto decimal de um dos mostradores de 7 segmentos)
- ✓ Leia o manual de referência da [placa Nexys2](#) ou da [placa Nexys](#) para entender como funcionam os mostradores
- ✓ Abra e estude o código VHDL contido no material de apoio disponível no link [dspl_drv_nexys.vhd](#). Deve-se acrescentar este arquivo no seu projeto
 - No ambiente ISE, ir na janela **Sources** (canto superior esquerdo), clique com botão direito do *mouse* no ícone com o nome do dispositivo (**xc3s1200E-4FG320 / xc3S200-4FT256**), escolhendo a opção de menu **Add Source**
- ✓ Modifique o arquivo UCF para adaptá-lo ao novo formato da saída e prototipar

PROJETO 2

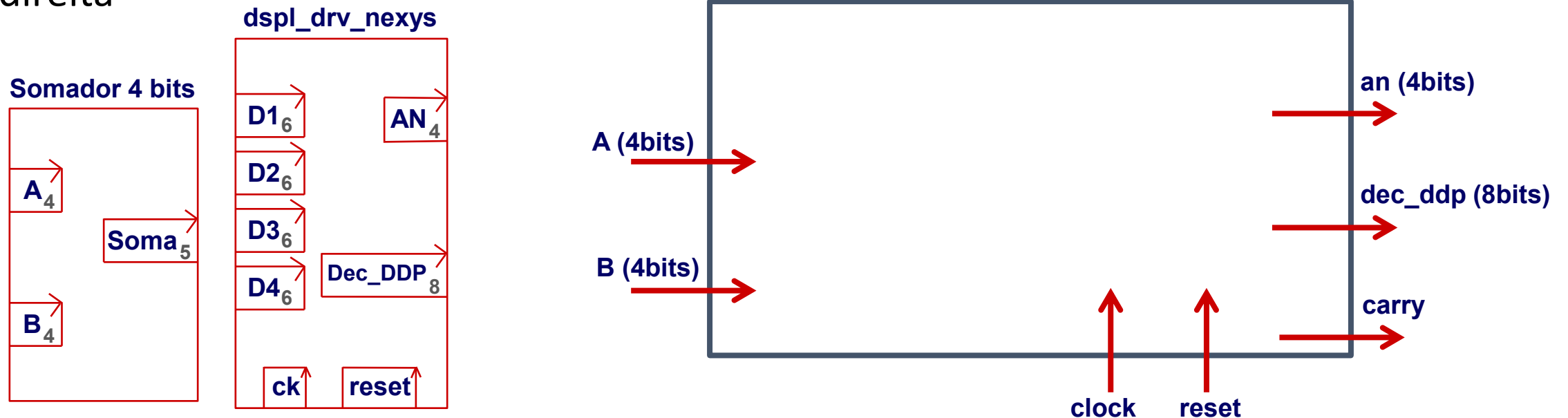
- Para ajudar na compreensão desta parte do trabalho, segue abaixo um diagrama parcial de blocos e conexões do circuito resultante. A expressão **E/A** corresponde a um par **Entidade/Arquitetura** que define um módulo de hardware descrito em VHDL



- ✓ Notem que se trata de uma descrição hierárquica: O somador e o driver são instanciados dentro da descrição VHDL da entidade, no topo da hierarquia de projeto (usando comandos **port map** em VHDL)
- ✓ Nem todos os pinos e fios estão nomeados explicitamente no diagrama. Complete-os no(s) arquivo(s) VHDL

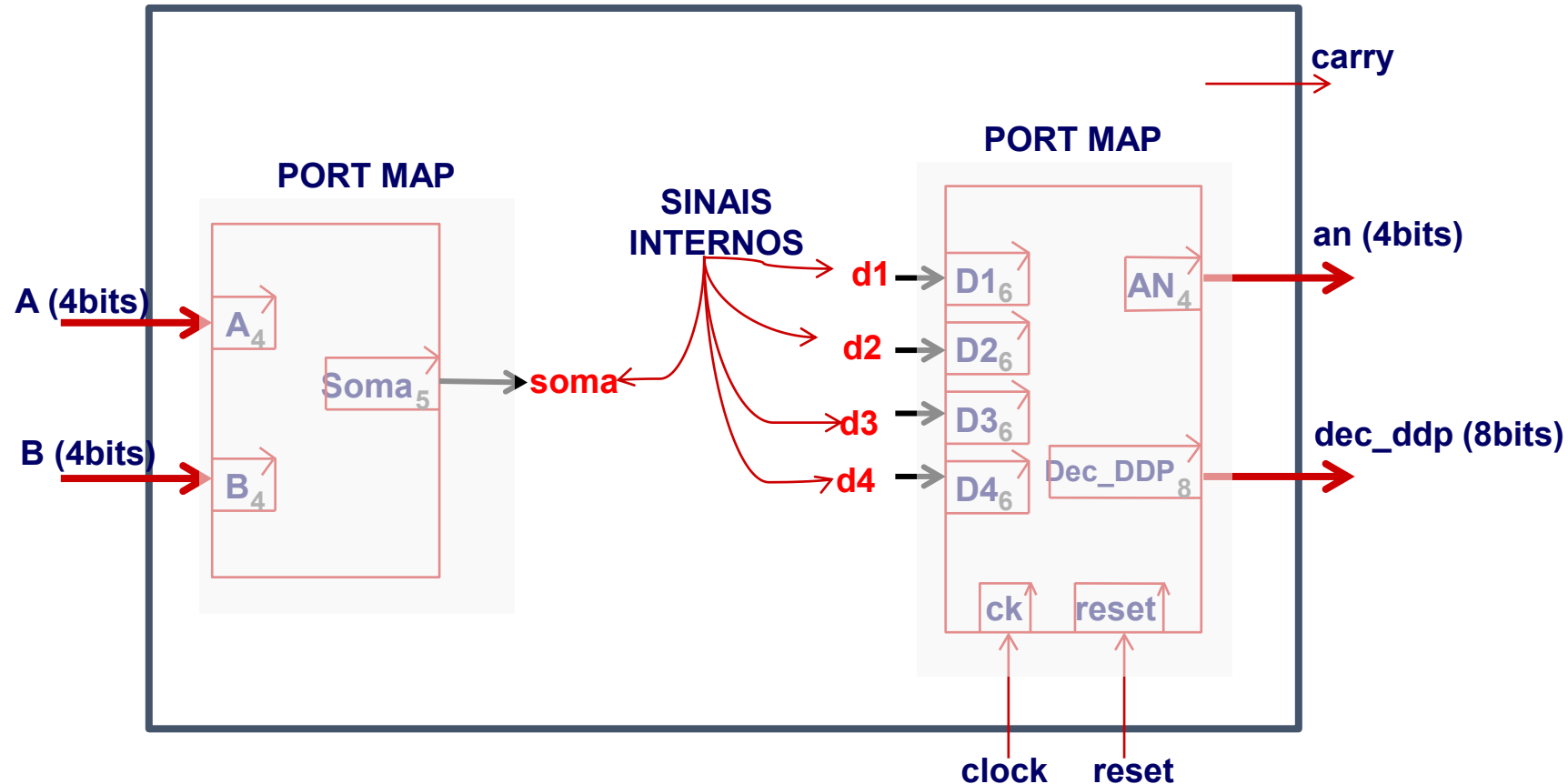
PROJETO 2

1. Colocar em um diretório [src](#) os fontes do somador gerados aqui e do acionador (*driver*) de display (disponível [aqui](#))
2. Adicionar no diretório [src](#) o arquivo UCF disponível [aqui](#) (para Nexys 2), que reflete a relação entre a entidade do módulo VHDL de mais alto nível hierárquico (top) e os pinos do FPGA conectados a dispositivos de entrada ou saída, conforme o desenho abaixo, à direita



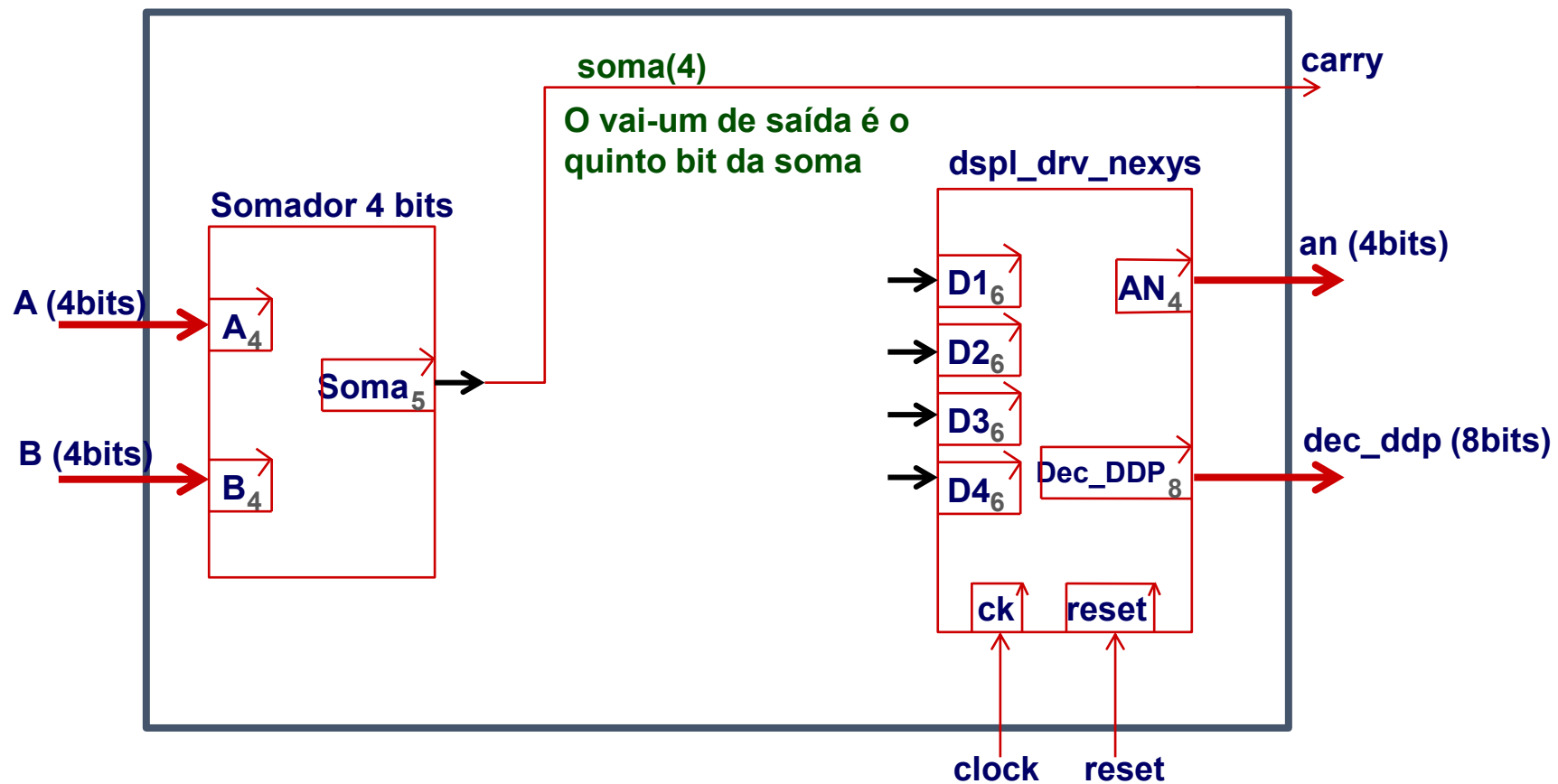
PROJETO 2

3. Deve-se criar um arquivo VHDL que faz o papel de topo da hierarquia de projeto e que será denominado `top.vhd`. Ao inserir neste arquivo os 2 comandos port map (que instanciam o somador e o driver do mostrador), alguns sinais internos precisam ser criados, ver abaixo



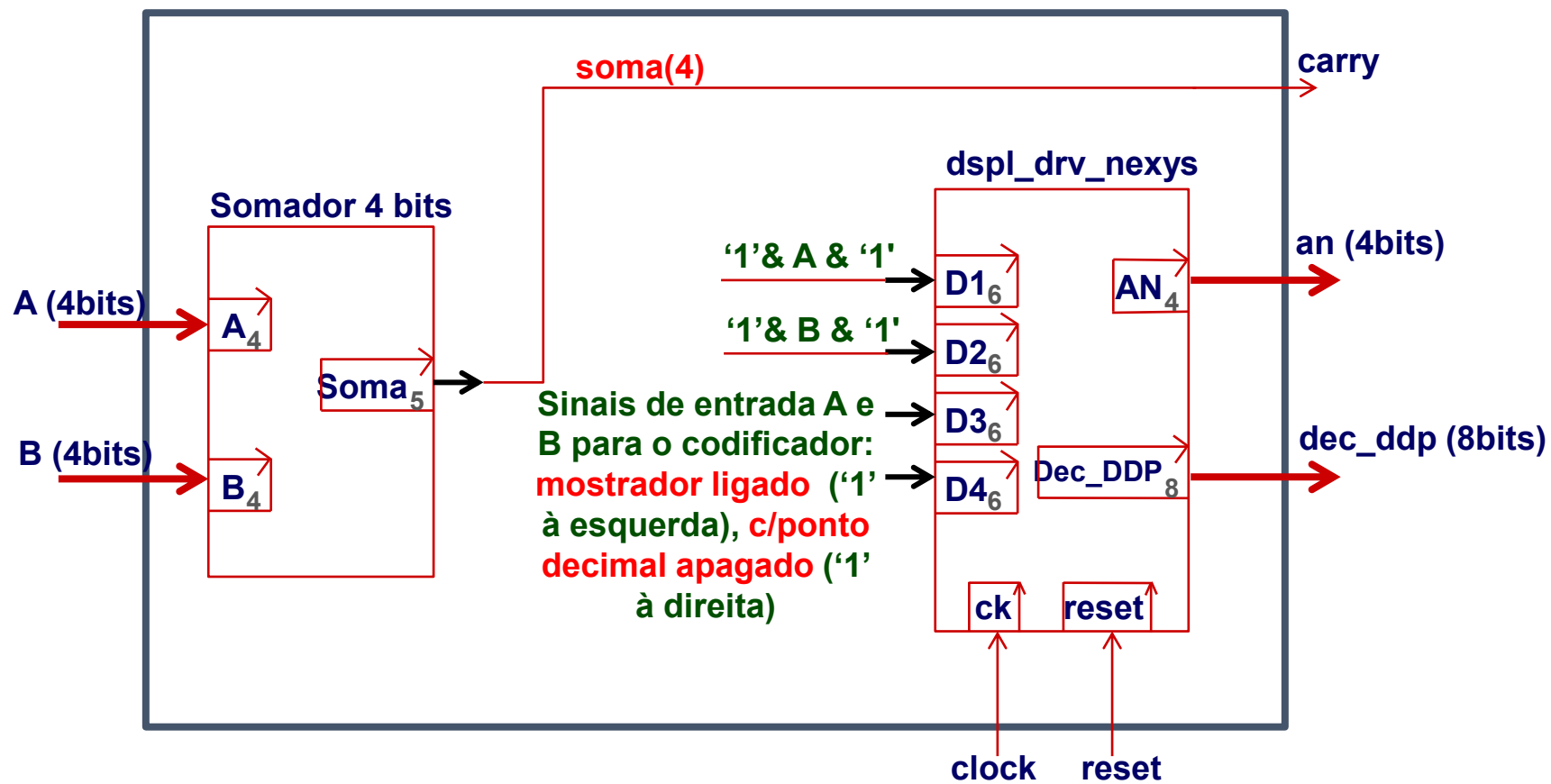
PROJETO 2

4. Ligar os sinais internos



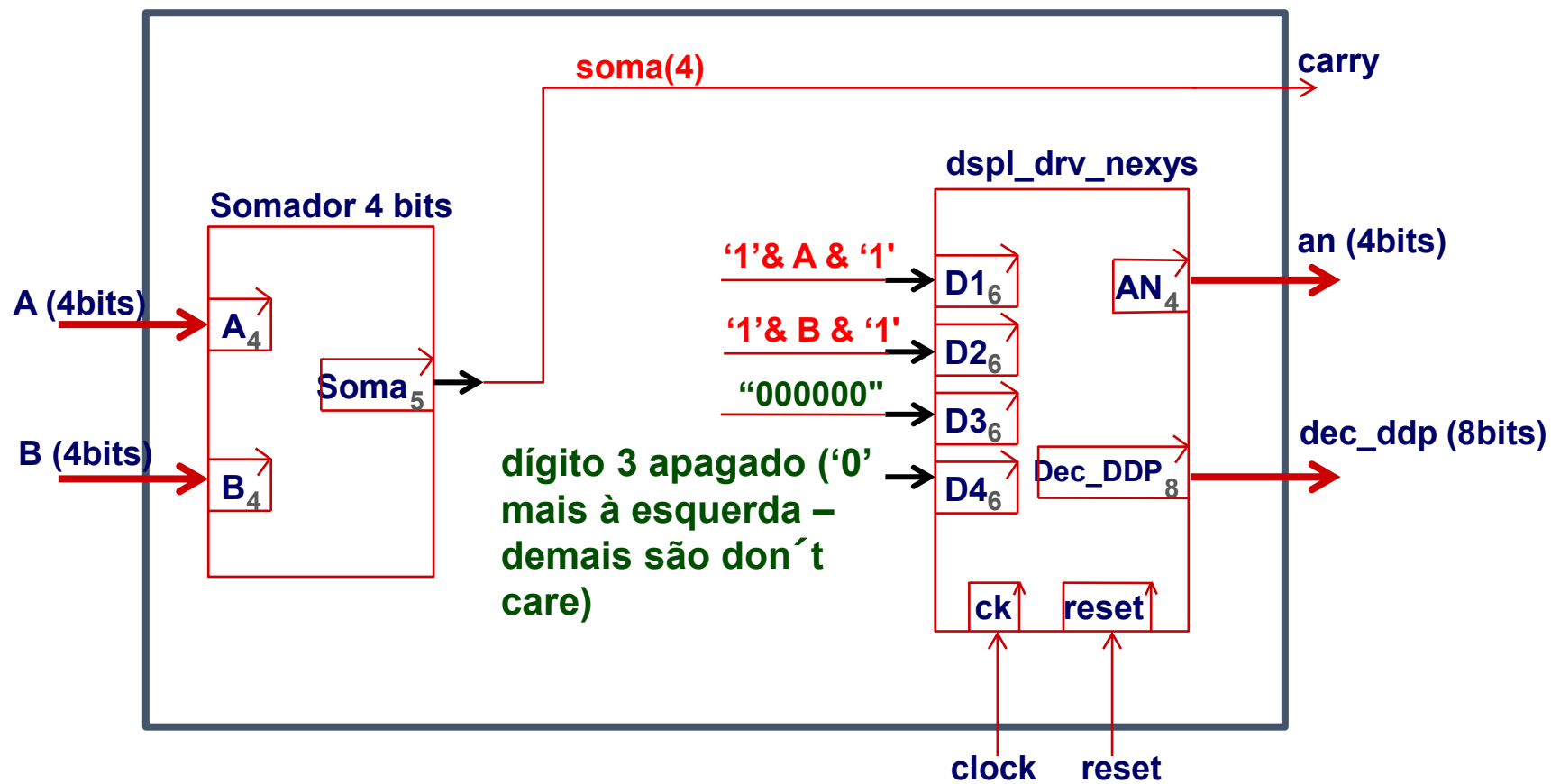
PROJETO 2

4. Ligar os sinais internos



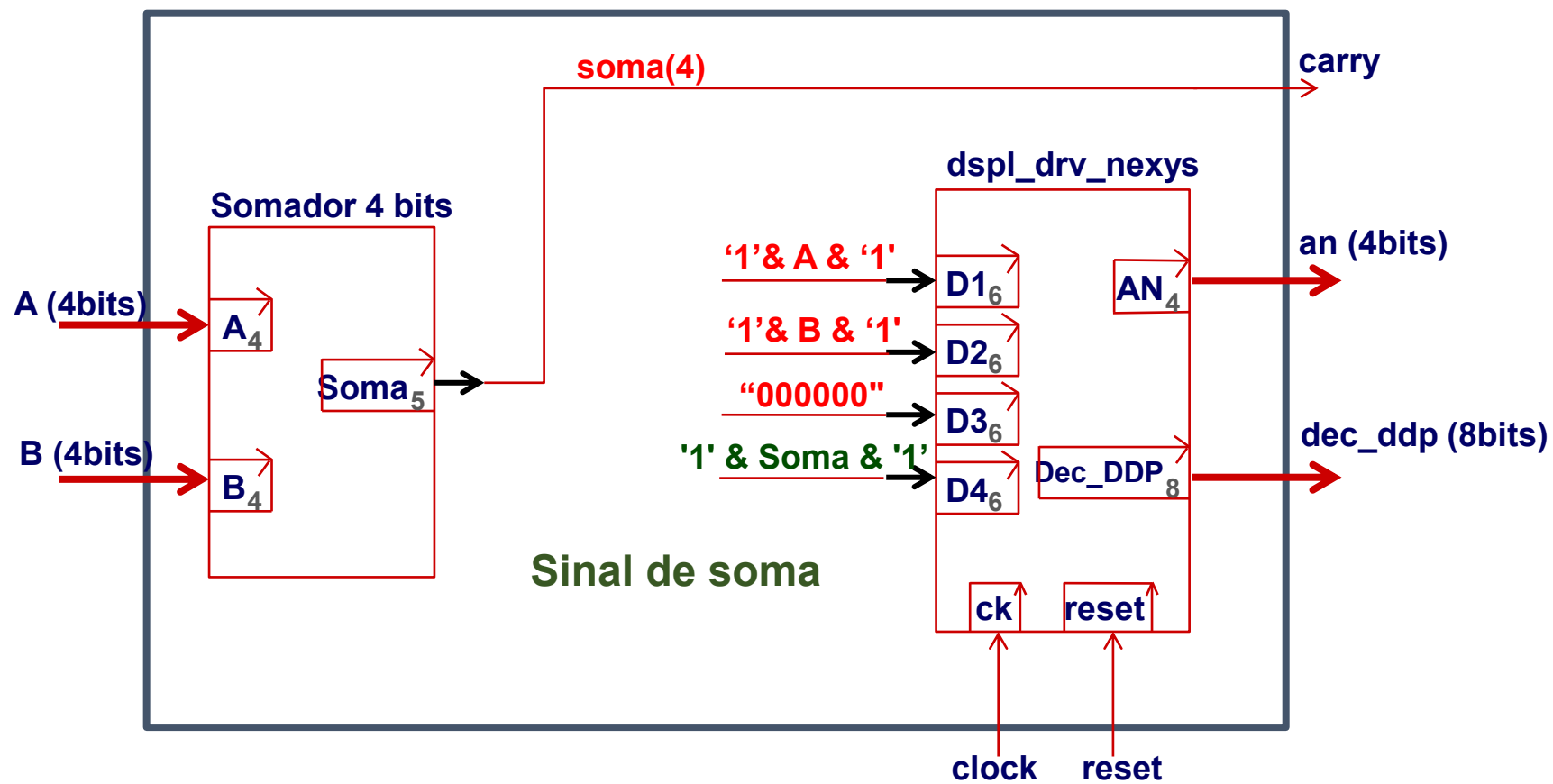
PROJETO 2

4. Ligar os sinais internos



PROJETO 2

4. Ligar os sinais internos



RESUMO DOS ADENDOS DO TUTORIAL 2

- O Resultado final do **Tutorial 2** consiste de (Arquivos de implementação serão disponibilizados, o relatório é deixado como tarefa aos discentes)
 - ✓ Um relatório em formato PDF, descrevendo o Projeto 1 e o Projeto 2, de acordo com o modelo apresentado
 - ✓ Um diretório para o Projeto 1, contendo os seguintes arquivos
 - VHDL do somador com vai-um
 - VHDL do testbench
 - arquivo .ucf do projeto
 - arquivo .bit do projeto
 - ✓ Um diretório para o Projeto 2, contendo os seguintes arquivos
 - VHDL do top.vhd com as instâncias do somador e do acionador (*driver*)
 - arquivo .ucf do projeto
 - arquivo .bit do projeto